

Fixed Income Analytics

Renato Antonio Palma Valencia

April 2026

Contents

1	Module 1: Bond Pricing	3
1.1	Introduction and Class Hierarchy	3
1.2	Definition and Fair Price	4
1.3	Generalization and Examples	5
1.4	Yield to Maturity (YTM)	6
1.5	Bond Parity	7
1.6	Duration & DV01	8
1.7	Convexity and Taylor Approximation	10
1.8	Conclusions and Key Takeaways	10
2	Module 2: Yield Curve	12
2.1	Introduction and Class Hierarchy	12
2.2	Term Structure of Interest Rates and Discount Factor	13
2.3	Forward Rates and Instantaneous Forward Rates	14
2.4	Bootstrapping	15
2.5	Curve Interpolation	15
2.6	Conclusions and Key Takeaways	18
3	Module 3: Sensitivity Analysis	20
3.1	Introduction and Class Hierarchy	20
3.2	Parallel Shift DV01	21
3.3	Key Rate Durations	22
3.4	Portfolio Sensitivity	24
3.5	Conclusions and Key Takeaways	25
4	Module 4: PCA on Term Structure	27
4.1	Introduction and Class Hierarchy	27
4.2	Motivation	28
4.3	PCA Framework	29
4.4	The Three Factors (Level, Slope and Curvature)	30
4.5	PCA Reconstruction and Risk Management	32
4.6	Conclusions and Key Takeaways	34
5	Module 5: IRS Pricing	36
5.1	Introduction and Class Hierarchy	36
5.2	Interest Rate Swap Definition and Structure	37
5.3	The Floating Leg equals Par	38
5.4	Swap as a Portfolio of FRAs	38
5.5	Swap Rate K^*	39
5.6	Swap Value at t	40
5.7	Swap DV01 and Applications	40
5.8	Conclusions and Key Takeaways	42
6	Project Summary and Further Research	43

Abstract

This document develops the mathematical foundations of Fixed Income Analytics implemented in the companion Python library `fixed-income-analytics`. Each module corresponds directly to a Python module in `/src`.

1 Module 1: Bond Pricing

Python Implementation

Module: `src/bonds.py`

Class: `VanillaBond`

Notebook: `notebooks/01.bond-pricing.ipynb`

1.1 Introduction and Class Hierarchy

This module implements the mathematical foundations of vanilla bond pricing. Starting from the definition of a bond as a sequence of deterministic cash flows, we derive closed-form pricing formulas under discrete, generalized and continuous compounding, introduce the yield-to-maturity as the internal rate of return of the instrument, and develop the first and second-order risk metrics that form the language of fixed income trading.

```
src/bonds.py
|- VanillaBond
    |-- Pricing
    |   |-- cash_flows()
    |   |-- bond_price(ytm)
    |   |-- bond_price_continuous(r)
    |-- Yield
    |   |-- ytm(market_price)
    |-- Risk Metrics
    |   |-- risk_metrics(ytm)
    |       |-- Macaulay Duration
    |       |-- Modified Duration
    |       |-- Convexity
    |       |-- DV01
    |-- Analysis
    |   |-- price_change(ytm, dy)
    |   |-- parity(ytm)
    |-- Validation
    |   |-- validate_roundtrip(r)
```

In `bonds.py` the entire module is encapsulated in a single class `VanillaBond`. A bond is instantiated with its economic parameters and all pricing, risk and analysis methods are exposed as instance methods, allowing clean and readable usage:

```

1  bond = VanillaBond(coupon_rate = 0.05,
2                      face_value = 1000,
3                      maturity = 10,
4                      frequency = 1)
5
6  # Pricing
7  P = bond.bond_price(ytm = 0.04)
8
9  # Risk Metrics
10 metrics = bond.risk_metrics(ytm = 0.04)
11
12 # Taylor Approximation
13 approx = bond.price_change(ytm = 0.04, dy = 0.01)

```

1.2 Definition and Fair Price

Definition 1.1 (Vanilla Bond). A bond is a fixed income instrument that represents a debt obligation. The issuer commits to pay:

1. Coupons C at periodic dates $t_1, t_2, \dots, t_n$.
2. A principal (Face Value) F at maturity date t_n .

The cash flow at each period is formally defined as:

$$CF_i = \begin{cases} C & i = 1, 2, \dots, n-1 \\ C + F & i = n \end{cases}$$

Proposition 1.1 (Bond Fair Price). *The fair price P of a bond is given by:*

$$P = \frac{C(1 - (1+r)^{-n})}{r} + F(1+r)^{-n}$$

with r the constant periodic discount rate and n the number of periods.

Proof. Notice that the price of a bond equals to the present value of all its cash flows:

$$P = \sum_{i=1}^n \frac{C}{(1+r)^i} + \frac{F}{(1+r)^n} = \frac{F}{(1+r)^n} + C \sum_{i=1}^n \left(\frac{1}{1+r} \right)^i$$

Using the geometric sum formula $\sum_{i=1}^n q^i = \frac{1-q^{n+1}}{1-q}$ with $q = 1/(1+r)$:

$$P = \frac{F}{(1+r)^n} + C \left[\frac{1 - \left(\frac{1}{1+r} \right)^{n+1}}{1 - \frac{1}{1+r}} - 1 \right] = F(1+r)^{-n} + C \left[\frac{(1+r) - (1+r)^{-n} - r}{r} \right]$$

$$P = C \left[\frac{1 - (1+r)^{-n}}{r} \right] + F(1+r)^{-n}$$

□

1.3 Generalization and Examples

If the periodic discount rate r compounds T times per period, the balance factor is: $(1 + \frac{r}{T})^{nT}$ and the price generalizes to:

Proposition 1.2. *The fair price P of a bond is:*

$$P = C \cdot \left[\frac{1 - (1 + \frac{r}{T})^{-nT}}{(1 + \frac{r}{T})^T - 1} \right] + F \left(1 + \frac{r}{T}\right)^{-nT}$$

with r the periodic discount rate, n the maturity, T the number of compounds of the discount rate per period.

Proof.

$$\begin{aligned} P &= \sum_{i=1}^n \frac{C}{(1 + \frac{r}{T})^{iT}} + \frac{F}{(1 + \frac{r}{T})^{nT}} = \frac{F}{(1 + \frac{r}{T})^{nT}} + C \sum_{i=1}^n \frac{1}{(1 + \frac{r}{T})^{iT}} \\ &= \frac{F}{(1 + \frac{r}{T})^{nT}} + C \sum_{i=1}^n \left(\left(\frac{1}{1 + \frac{r}{T}} \right)^T \right)^i = \frac{F}{(1 + \frac{r}{T})^{nT}} + C \left[\frac{1 - \left(\frac{1}{1 + \frac{r}{T}} \right)^{(n+1)T}}{1 - \left(\frac{1}{1 + \frac{r}{T}} \right)^T} - 1 \right] \\ &= \frac{F}{(1 + \frac{r}{T})^{nT}} + C \left[\frac{(1 + \frac{r}{T})^T - (1 + \frac{r}{T})^{-nT}}{(1 + \frac{r}{T})^T - 1} - 1 \right] = \frac{F}{(1 + \frac{r}{T})^{nT}} + C \left[\frac{1 - (1 + \frac{r}{T})^{-nT}}{(1 + \frac{r}{T})^T - 1} \right] \end{aligned}$$

□

The general formula unifies several classical instruments as particular cases:

1. **Zero-Coupon Bond** ($C = 0$): When the bond pays no coupons, the price reduces to:

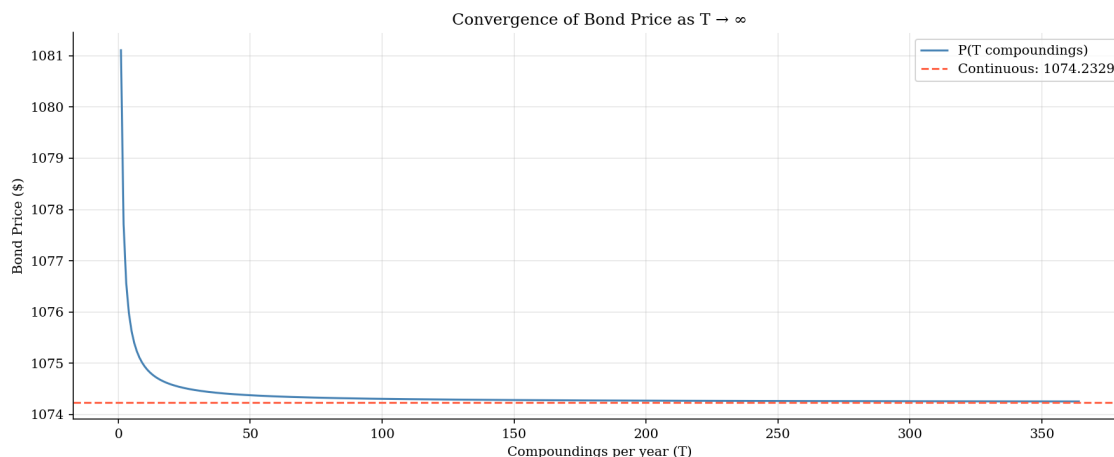
$$P = F \left(1 + \frac{r}{T}\right)^{-nT}$$

The price is simply the present value of the face value. This is the purest form of a fixed income instrument and plays a central role in bootstrapping, since each zero-coupon bond defines exactly one point on the discount curve. Note also that the Macaulay Duration of a zero coupon bond equals exactly its maturity: $D_{\text{mac}} = n$, which is the maximum possible duration for a bond of that maturity.

2. **Continuous Compounding** ($T \rightarrow \infty$): Taking $T \rightarrow \infty$ and using that $\lim_{T \rightarrow \infty} (1 + \frac{r}{T})^{nT} = e^{rn}$, we get the following price of the bond:

$$P = C \cdot \left(\frac{1 - e^{-rn}}{e^r - 1} \right) + F e^{-rn}$$

this is the standard continuous compounding formula used throughout quantitative finance and is the natural limit of the general formula. Consider the case of a bond with a coupon rate $C/F = 5\%$, a face value $F = \$1000$, a maturity $n = 10$ and a discount rate $r = 0.04$. Notice in Figure 1 how the price converges as there are more compounds T per year, this is, as $T \rightarrow \infty$:

Figure 1: Convergence of Bond Price as $T \rightarrow \infty$

3. **Consol Bond** ($n \rightarrow \infty$): A consol (or perpetuity) pays coupons forever and never returns the principal. Taking $n \rightarrow \infty$ in the price formula and noting that $(1 + r)^{-nT} \rightarrow 0$:

$$P = \frac{C}{\left(1 + \frac{r}{T}\right)^T - 1}$$

The price is simply the coupon divided by the discount balance factor. This result has a beautiful intuition: A perpetuity is worth exactly the amount you would need to invest at rate $\left(1 + \frac{r}{T}\right)^T - 1$ to generate coupon C forever. Note that the British government issued consols (Consolidated Annuities) that traded in market for over 200 years.

4. **Par Bond** ($P = F$): When the discount balance factor is $\left(1 + \frac{r}{T}\right)^T - 1 = \frac{C}{F}$, the price of the bond is:

$$P = C \cdot \left(\frac{1 - \left(1 + \frac{r}{T}\right)^{-nT}}{C/F} \right) + F \left(1 + \frac{r}{T}\right)^{-nT} = F - F \left(1 + \frac{r}{T}\right)^{-nT} + F \left(1 + \frac{r}{T}\right)^{-nT} = F$$

This bond trades at par. This case connects directly to the YTM parity result in section 1.4

1.4 Yield to Maturity (YTM)

Definition 1.2 (Yield to Maturity). The YTM is the rate y that equates the market price of the bond with the present value of its cash flows:

$$P = \sum_{i=1}^n \frac{CF_i}{\left(1 + \frac{y}{T}\right)^{iT}}$$

This equation has no closed-form solution and must be solved numerically.

Definition 1.3 (Newton Raphson Method). Define the objective function:

$$f(y) = \sum_{i=1}^n \frac{CF_i}{(1 + \frac{y}{T})^{iT}} - P = 0$$

and its derivative, used for the Newton Step:

$$f'(y) = - \sum_{i=1}^n \frac{i \cdot CF_i}{(1 + \frac{y}{T})^{iT+1}}$$

The Newton-Raphson iteration updates the estimate with the following recursion:

$$y_{k+1} = y_k - \frac{f(y_k)}{f'(y_k)}$$

starting from an initial guess y_0 .

The algorithm converges quadratically to the true YTM. Convergence is guaranteed because $f(y)$ is strictly convex and monotonically decreasing in y , which ensures a unique root. In `bonds.py` this is implemented via `scipy.optimize.newton` with analytical derivative for maximum efficiency. The key price-yield relationship is:

$$\uparrow y \Rightarrow \downarrow P \quad \& \quad \downarrow y \Rightarrow \uparrow P$$

1.5 Bond Parity

Proposition 1.3. Let $c = C/F$ be the coupon rate. Then:

$$P \leq F \iff \left(1 + \frac{y}{T}\right)^T - 1 \geq c$$

Proof. Substituting $\left(1 + \frac{y}{T}\right)^T - 1 = c$ and $C = cF$:

$$P = cF \cdot \left(\frac{1 - \left(1 + \frac{y}{T}\right)^{-nT}}{c}\right) + F \left(1 + \frac{y}{T}\right)^{-nT} = F - F \left(1 + \frac{y}{T}\right)^{-nT} + F \left(1 + \frac{y}{T}\right)^{-nT} = F$$

Since P is strictly decreasing in y ($f'(y) < 0$) and $\left(1 + \frac{y}{T}\right)^T - 1 > 0$ then the result follows directly. $\square$

When there is one capitalization per year ($T = 1$) the condition collapses to:

$$P \leq F \iff y \geq c$$

The economic intuition for this case is clear: when the market demands a return $y > c$, the bond's coupons are insufficient, so the market discounts the price below par to compensate with a capital gain at maturity. The opposite holds when $y < c$. To see this graphically, consider the case of a bond with coupon rate $c = C/F = 5\%$, a face value of \$1000, a maturity $n = 10$ and a discrete capitalization $T = 1$. The Figure 2 plots the price of the bond P against the yield y and shows the areas where the bond is at premium or discount.

Regardless of whether a bond starts at premium or discount, its price will converge to its face value F as it approaches maturity ($t \rightarrow n$), a phenomenon known as *Pull to Par*.

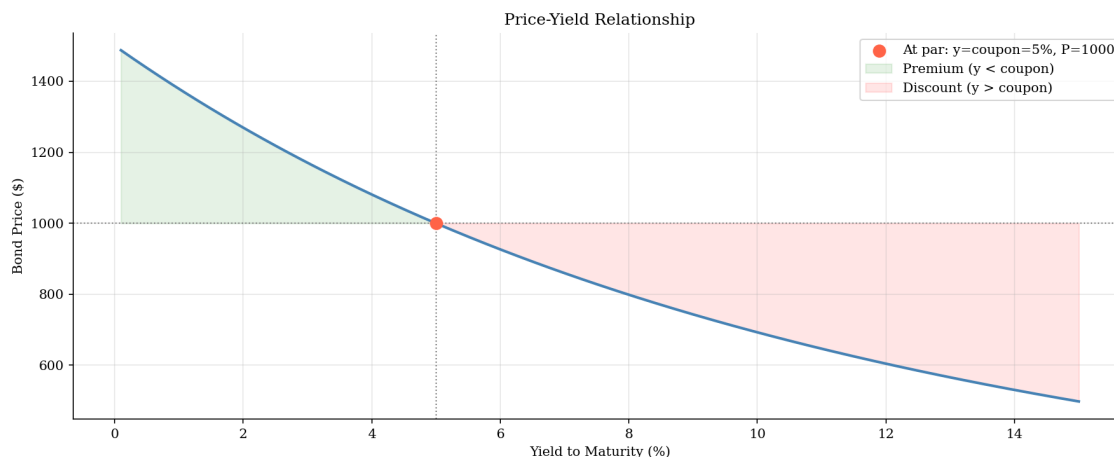


Figure 2: Price-Yield Relationship

Condition	Name	Intuition
$P = F \iff y = c$	Bond at Par	Market rate equals coupon rate
$P > F \iff y < c$	Bond above Par (Premium Bond)	Generous coupons, price rises
$P < F \iff y > c$	Bond below Par (Discount Bond)	Insufficient coupons, price drops

Table 1: Bonds Parity

1.6 Duration & DV01

Definition 1.4 (Macaulay Duration). Macaulay Duration is the weighted average time of cash flows of the bond, where weights are present values:

$$D_{\text{mac}} = \frac{1}{P} \sum_{i=1}^n i \cdot \frac{CF_i}{\left(1 + \frac{y}{T}\right)^{iT}}$$

where CF_i is the cash flow at period i and T the number of coupons periods per year. This is the average time for the investor to recover the investment.

As we said in section 1.3, the zero-coupon bond has a Macaulay Duration $D_{\text{mac}} = n$, now we can prove it by applying the definition and recalling that $P_{\text{zero}} = F \left(1 + \frac{y}{T}\right)^{-nT}$:

$$D_{\text{mac}} = F^{-1} \left(1 + \frac{y}{T}\right)^{nT} \cdot nF \left(1 + \frac{y}{T}\right)^{-nT} = n$$

Definition 1.5 (Modified Duration). It measures the price sensitivity to yield changes:

$$D_{\text{mod}} = \frac{D_{\text{mac}}}{1 + \frac{y}{T}}$$

We can plot these durations against maturity to see how they behave. Consider a bond with four different coupon rates $c = C/F = (0\%, 2\%, 5\%, 10\%)$, face value $F = 1000$ and yield $y = 5\%$

that compounds once per year $T = 1$. Figure 3 shows durations against maturity and how as coupon rates increase, duration gets lower.

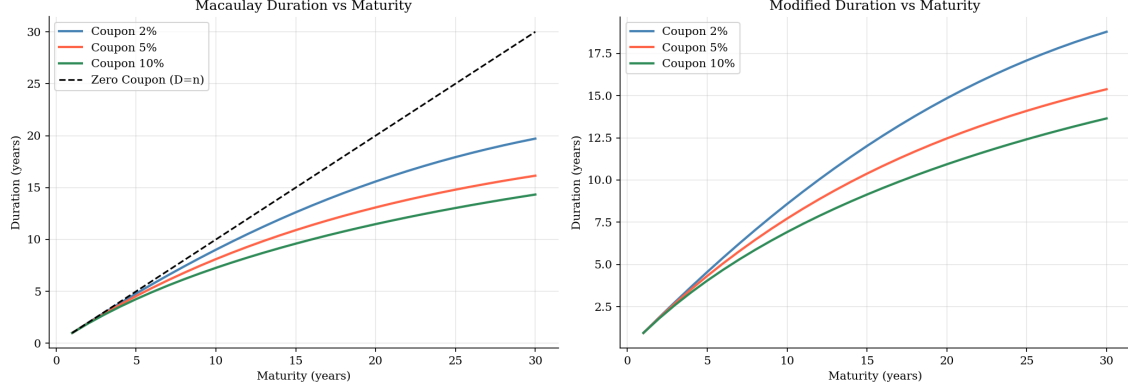


Figure 3: Durations versus Maturity

Proposition 1.4. *Price sensitivity can be written as:*

$$\frac{dP}{dy} = -D_{mod} \cdot P$$

Proof. Remember that:

$$P = \sum_{i=1}^n \frac{C}{(1 + \frac{y}{T})^{iT}} + \frac{F}{(1 + \frac{y}{T})^{nT}} = \sum_{i=1}^n \frac{CF_i}{(1 + \frac{y}{T})^{iT}}$$

$$\frac{dP}{dy} = - \sum_{i=1}^n \frac{i \cdot CF_i}{(1 + \frac{y}{T})^{iT+1}} = - \frac{1}{1 + \frac{y}{T}} \sum_{i=1}^n i \cdot \frac{CF_i}{(1 + \frac{y}{T})^{iT}} = -P \cdot \frac{D_{mac}}{1 + \frac{y}{T}} = -D_{mod} \cdot P$$

□

In continuous compounding ($T \rightarrow \infty$):

$$D_{mac} = \lim_{T \rightarrow \infty} D_{mod} \left(1 + \frac{y}{T}\right) = D_{mod} = -\frac{1}{P} \frac{dP}{dy}$$

Definition 1.6 (Dollar Value of a Basis Point). DV01 is how much the bond price changes when the yield changes 1 basis point (0.01%):

$$DV01 = -\frac{dP}{dy} \cdot 0.0001 = D_{mod} \cdot P \cdot 0.0001$$

1.7 Convexity and Taylor Approximation

As duration is a first order linear approximation, convexity captures the second order effect:

Definition 1.7 (Bond Convexity). Bond convexity measures the non-linear relationship between price and yield:

$$\mathcal{C} = \frac{1}{P} \frac{d^2 P}{dy^2} = \frac{1}{P} \sum_{i=1}^n \frac{i(iT+1)}{T} \cdot \frac{CF_i}{(1 + \frac{y}{T})^{iT+2}}$$

Combining Duration and Convexity gives the second-order Taylor approximation of price change:

$$P(y) = P(y_0) + \frac{dP}{dy}(y - y_0) + \frac{1}{2} \frac{d^2 P}{dy^2}(y - y_0)^2 + \dots$$

$$P(y) - P(y_0) \approx -D_{\text{mod}} \cdot P(y - y_0) + \frac{1}{2} P \cdot \mathcal{C}(y - y_0)^2$$

$$\Delta P \approx P \left(-D_{\text{mod}} \cdot \Delta y + \frac{1}{2} \mathcal{C} \cdot (\Delta y)^2 \right)$$

Convexity is always positive for vanilla bonds, meaning the price-yield curve is convex: gains from falling yields exceed losses from equal rising yields. This is a desirable property for the bond-holder and explains why higher convexity bonds command a premium in the market.

We can visualize the second-order Taylor approximation and the gains of adding the second-order term (convexity) instead of just approximating with the first order term for a bond with the same characteristics as the examples above.

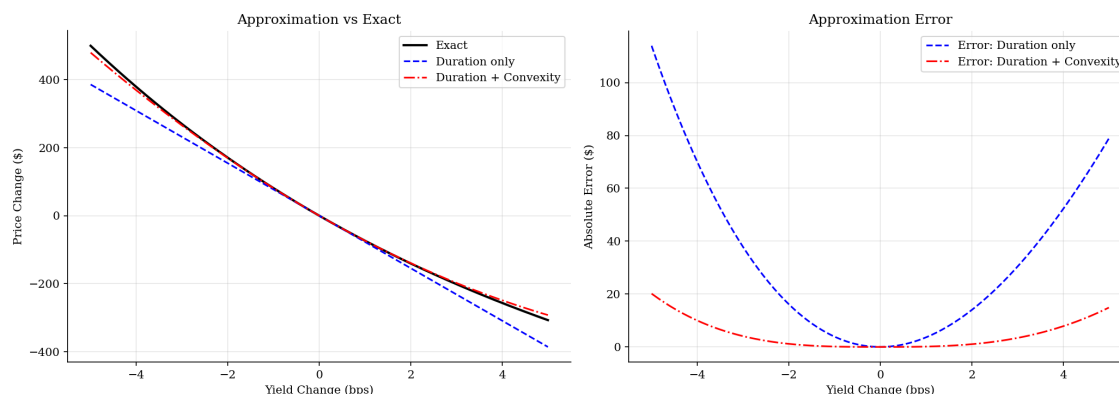


Figure 4: First and Second-Order Approximation of a Bond Price

At changes of the yield of 100 bp, the approximation error with the first-order term only is \$3.4751, adding the second-order term we get an absolute error of \$0.1256.

1.8 Conclusions and Key Takeaways

- The bond price is a strictly convex, decreasing function of the yield: Duration captures the slope and Convexity captures the curvature of this relationship.

- The YTM has no closed-form solution but converges quadratically under Newton-Raphson, since the objective function is strictly convex and monotone.
- Convexity is always positive for vanilla bonds: the second-order Taylor approximation materially improves the price estimate for yield moves above 50 bps.

The bond pricing framework of this module rests on a critical simplification: the discount rate r is assumed constant across all maturities. In practice, this is never true. A 1-year investment and a 10-year investment command different rates, and the relationship between rates and maturities (the *term structure* of interest rates) is one of the most important objects in fixed income. Formally, replacing the constant r with a maturity dependent function $r(t)$ transforms the bond price from:

$$P = \sum_{i=1}^n \frac{CF_i}{(1 + \frac{r}{T})^{-iT}}$$

into:

$$P = \sum_{i=1}^n \frac{CF_i}{(1 + \frac{r(t_i)}{T})^{-t_i T}} \xrightarrow{\text{In Continuous Capitalization}} P = \sum_{i=1}^n CF_i e^{-r(t_i) \cdot t_i}$$

where each cash flow is discounted at its own maturity-specific rate. This requires building the entire function $r(t)$ from observed market prices, the problem of **yield curve construction**, which is the subject of module [2](#).

2 Module 2: Yield Curve

Python Implementation

Module: `src/curves.py`

Classes: `DiscountCurve`, `Bootstrapper` and `CurveInterpolator`

Notebook: `notebooks/02_yield_curve.ipynb`

2.1 Introduction and Class Hierarchy

This module extends the single-rate bond pricing of Module 1 to the full term structure setting. Rather than discounting all cash flows at a constant rate r , we build a continuous function $r(t)$ from observable market prices, interpolate it across arbitrary maturities, and fit a parametric model that captures its economic structure. The three classes in `src/curves.py` handle these three tasks in sequence.

```
src/curves.py
|
|-- DiscountCurve
|   |
|   |-- Attributes
|   |   |-- maturities      Observed tenors
|   |   |-- spot_rates      Continuously compounded
|   |
|   |-- Core
|   |   |-- discount_factor(t)       $Z(t) = e^{(-r(t)*t)}$ 
|   |   |-- discount_factors()      Z vector
|   |   |-- spot_rate(t)            Linear interp
|   |
|   |-- Rates
|   |   |-- forward_rate(t1, t2)     $f(t1, t2)$ 
|   |   |-- instantaneous_forward(t)  $f(t)$  numerical
|   |   |-- forward_curve(t_grid)    $f(t)$  over grid
|   |
|-- Bootstrapper
|   |-- bootstrap(prices, cash_flows, Sequential
|   |               maturities)          extraction
|   |-- from_yields(maturities, yields) FRED/market data
|   |
|-- CurveInterpolator
|   |-- linear(t)                Linear on zero rates
|   |-- cubic_spline(t)          Smooth, C2 continuous
|   |-- cubic_spline_forward(t)   $f(t)$  via spline deriv
|   |-- nss_spot(t, beta)        NSS spot formula
|   |-- nss_forward(t, beta)     NSS forward formula
|   |-- fit_nss()                L-BFGS-B calibration
```

```

|-- nss_summary(beta)      Economic interpretation
|-- compare(t_grid, beta)  All three methods

```

The three classes are designed to be used in sequence. `DiscountCurve` is the fundamental data structure, holding observed spot rates and computing discount factors. `Bootstrapper` constructs a `DiscountCurve` from raw market data. `CurveInterpolator` takes a `DiscountCurve` and extends it to arbitrary maturities via three methods of increasing sophistication:

```

1  # Step 1: Build curve from FRED yields
2  curve = Bootstrapper.from_yields(maturities, yields)
3
4  # Step 2: Interpolate at arbitrary maturities
5  interp = CurveInterpolator(curve)
6
7  # Step 3: Fit NSS parametric model
8  beta = interp.fit_nss()
9  r_nss = CurveInterpolator.nss_spot(t_grid, beta)

```

2.2 Term Structure of Interest Rates and Discount Factor

In the bond pricing of Module 1 we assumed a constant discount rate r . In reality, rates vary with maturity: a 1-year investment typically commands a different rate than a 10-year one.

Definition 2.1 (Term Structure of Interest Rates). The term structure of interest rates is a function $r(t)$ that assigns a spot rate to each maturity t .

Assuming that $r(t)$ is continuously differentiable, the bond price generalizes to:

$$P = \sum_{i=1}^n CF_i \cdot e^{-r(t_i)t_i}$$

where each cash flow is discounted at its own maturity-specific rate. This is the foundation of all modern fixed income pricing.

Definition 2.2 (Discount Factor). The continuous discount factor for maturity t is:

$$Z(t) = e^{-r(t) \cdot t}$$

It represents the present value today of \$1 to be received at time t . The bond price can be written compactly as:

$$P = \sum_{i=1}^n CF_i \cdot Z(t_i)$$

Discount factors have three fundamental properties. First $Z(0) = 1$: a dollar today is worth a dollar. Second, $Z(t) \in (0, 1), \forall t > 0$ and positive rates: future cash flows are always worth less than their face value today. Third, $Z(t)$ is decreasing in t for positive rates: the further in the future, the less a dollar is worth today.

The spot rate can always be recovered from the discount factor:

$$r(t) = -\frac{\ln Z(t)}{t}$$

2.3 Forward Rates and Instantaneous Forward Rates

Definition 2.3 (Forward Rate). The forward rate $f(t_1, t_2)$ is the implicit rate agreed today for an investment between two future dates t_1 and t_2 . It is defined by the no-arbitrage condition:

$$e^{r(t_2) \cdot t_2} = e^{r(t_1) \cdot t_1} \cdot e^{f(t_1, t_2) \cdot (t_2 - t_1)}$$

Applying logarithm at both sides:

$$\begin{aligned} r(t_2)t_2 &= r(t_1)t_1 + f(t_1, t_2)(t_2 - t_1) \\ f(t_1, t_2) &= \frac{r(t_2)t_2 - r(t_1)t_1}{t_2 - t_1} \end{aligned}$$

The economic intuition is clear: investing from 0 to t_2 at spot rate $r(t_2)$ must yield the same as investing from 0 to t_1 at $r(t_1)$ and then rolling over from t_1 to t_2 at the forward rate $f(t_1, t_2)$. Otherwise there would be an arbitrage opportunity.

In terms of the discount factors, the forward rate has an equivalent and elegant expression:

$$f(t_1, t_2) = \frac{\ln Z(t_1) - \ln Z(t_2)}{t_2 - t_1}$$

Proposition 2.1. *The instantaneous forward rate is given by:*

$$f(t) = r(t) + t \cdot \frac{\partial r(t)}{\partial t}$$

Proof. Setting $t_1 = t$ and taking the limit $t_2 \rightarrow t$:

$$f(t) = \lim_{t_2 \rightarrow t} f(t, t_2) = \lim_{t_2 \rightarrow t} \frac{r(t_2) \cdot t_2 - r(t) \cdot t}{t_2 - t} = \frac{d}{dt}[r(t) \cdot t] = r(t) + t \cdot \frac{\partial r(t)}{\partial t}$$

□

The two terms have a clear interpretation. The first term $r(t)$ is simply the spot rate at maturity t . The second term $t \cdot \frac{\partial r(t)}{\partial t}$ is a correction for the slope of the yield curve: if the curve is upward sloping, the instantaneous forward rate lies above the spot rate, and vice versa. When the yield curve is flat ($\frac{\partial r(t)}{\partial t} = 0$), the instantaneous forward rate equals the spot rate exactly.

The inverse relationship allows recovering the spot rate from the forward curve:

$$t \cdot r(t) = \int_0^t f(s)ds \Rightarrow r(t) = \frac{1}{t} \int_0^t f(s)ds$$

The spot rate is the average of instantaneous forward rates up to maturity t . This is one of the most interesting results in fixed income: the entire spot curve is encoded in the forward curve and vice versa.

2.4 Bootstrapping

The yield curve is not directly observable. What we observe in the market are prices of traded bonds at specific maturities. Bootstrapping is the iterative procedure that extracts the unobservable spot rates from those observable prices.

Definition 2.4 (Bootstrapping). Bootstrapping is the process of extracting spot rates $r(t_1), \dots, r(t_n)$ sequentially from observed bond prices, using already extracted rates to solve for the next one.

Proposition 2.2. *Given the observed bond price P , cash flows CF_i and already extracted discount factors $Z(t_1), \dots, Z(t_{n-1})$, the spot rate at maturity n is:*

$$r(n) = \frac{1}{n} \cdot \ln \left(\frac{CF_n}{P - \sum_{i=1}^{n-1} CF_i \cdot Z(i)} \right)$$

Proof. Starting from the bond price equation under continuous compounding:

$$P = \sum_{i=1}^n CF_i \cdot e^{-ir(i)} = \sum_{i=1}^{n-1} CF_i \cdot Z(i) + CF_n \cdot Z(n)$$

Isolating $Z(n)$ and using that $Z(n) = e^{-r(n) \cdot n}$:

$$Z(n) = \frac{P - \sum_{i=1}^{n-1} CF_i \cdot Z(i)}{CF_n} = e^{-r(n) \cdot n}$$

Taking the logarithm and solving for $r(n)$:

$$r(n) = \frac{1}{n} \cdot \ln \left(\frac{CF_n}{P - \sum_{i=1}^{n-1} CF_i \cdot Z(i)} \right)$$

□

The procedure is sequential: $r(t_1)$ is extracted first (typically from a short-term instrument like a T-Bill), then $r(t_2)$ using the already known $Z(t_1)$ and so on. Each step uses all previously extracted rates, which is why the method is called bootstrapping: each rate "pulls up" the next one.

2.5 Curve Interpolation

Bootstrapping gives us spot rates only at the maturities of traded bonds, typically $\{0.25, 0.5, 1, 2, 3, 5, 7, 10, 20, 30\}$ years for the U.S. Treasuries. To price instruments at arbitrary maturities, we need to interpolate between these points.

Definition 2.5 (Linear Interpolation on Zero Rates). For $t \in [t_i, t_{i+1}]$, the linearly interpolated spot rate is:

$$r(t) = r(t_i) + \frac{r(t_{i+1}) - r(t_i)}{t_{i+1} - t_i} (t - t_i)$$

This is the simplest method: the spot rate moves linearly between observed nodes. It is easy to implement and understand, but has an important drawback: the implied forward rates are piecewise constant and exhibit discontinuous jumps at each node. In practice this means the forward curve has "kinks" at every tenor point, which is financially unrealistic.

Definition 2.6 (Cubic Spline Interpolation). A cubic spline fits a piecewise cubic polynomial $S(t)$ such that on each interval $[t_i, t_{i+1}]$:

$$S(t) = a_i + b_i(t - t_i) + c_i(t - t_i)^2 + d_i(t - t_i)^3$$

with the conditions that $S(t)$, $S'(t)$ and $S''(t)$ are all continuous at every interior node. This gives a system of $4(n - 1)$ unknowns with $4(n - 1)$ equations (matching values, first and second derivatives at each node, plus two boundary conditions).

The key advantage over linear interpolation is that the cubic spline produces a smooth forward curve. Since $f(t) = r(t) + t \cdot \frac{\partial r(t)}{\partial t}$, the forward rate depends on $\frac{\partial r(t)}{\partial t}$: a discontinuous $\frac{\partial r(t)}{\partial t}$ from linear interpolation generates a discontinuous forward curve, while the cubic spline guarantees $\frac{\partial r(t)}{\partial t}$ is continuous everywhere.

The two standard boundary conditions used in practice are:

1. **Natural Spline:** $S''(t_1) = S''(t_n) = 0$, meaning no curvature at the endpoints. Simple and the most common choice.
2. **Not-a-Knot:** forces $S'''(t)$ to be continuous at the second and second-to-last nodes. Generally produces a better behavior at the boundaries and is the default in `scipy.interpolate.CubicSpline`.

Proposition 2.3. *Linear interpolation on zero rates implies piecewise constant forward rates with jumps at nodes. Cubic spline interpolation implies continuous forward rates everywhere.*

Proof. Recall the instantaneous forward rate formula $f(t) = r(t) + t \cdot r'(t)$. Under linear interpolation we have that:

$$r'(t) = \frac{r(t_{i+1}) - r(t_i)}{t_{i+1} - t_i}$$

is constant on each interval but jumps at nodes, so $f(t)$ also jumps. Under cubic spline, $r'(t)$ is continuous by construction, so $f(t)$ is continuous everywhere. $\square$

In `curves.py` both methods are implemented and compared. The cubic spline is the default for all downstream calculations.

Both linear interpolation and cubic splines are non-parametric methods: they fit the observed data without imposing any financial structure on the curve. The Nelson-Siegel-Svensson model takes a fundamentally different approach: it parametrizes the entire yield curve with a small number of economically interpretable parameters.

Definition 2.7 (Nelson-Siegel-Svensson Model). The model starts with the instantaneous forward rate, which is specified as:

$$f(t) = \beta_0 + \beta_1 e^{-t/\lambda_1} + \beta_2 \frac{t}{\lambda_1} e^{-t/\lambda_1} + \beta_3 \frac{t}{\lambda_2} e^{-t/\lambda_2}$$

Integrating $r(t) = \frac{1}{t} \int_0^t f(s) ds$ (the result from section 2.3), the spot rate curve is:

$$r(t) = \beta_0 + \beta_1 \frac{1 - e^{-t/\lambda_1}}{t/\lambda_1} + \beta_2 \left(\frac{1 - e^{-t/\lambda_1}}{t/\lambda_1} - e^{-t/\lambda_1} \right) + \beta_3 \left(\frac{1 - e^{-t/\lambda_2}}{t/\lambda_2} - e^{-t/\lambda_2} \right)$$

with parameters $\beta = (\beta_0, \beta_1, \beta_2, \beta_3, \lambda_1, \lambda_2)$.

The first three terms constitute the original [Nelson and Siegel \(1987\)](#) model; the fourth term is the [Svensson \(1994\)](#) extension that adds a second hump to accommodate more complex curve shapes.

The parameters have direct economic interpretations. Taking limits:

$$\lim_{t \rightarrow \infty} r(t) = \beta_0 \quad \lim_{t \rightarrow 0} r(t) = \beta_0 + \beta_1$$

so β_0 is the long-run level of interest rates and $\beta_0 + \beta_1$ is the short-run level. The parameters β_2 and β_3 control the magnitude of the humps at intermediate maturities, with λ_1 and λ_2 controlling where those humps are located. For a normal upward-sloping curve, $\beta_1 < 0$ (short rates below long rates) and $\beta_2 > 0$ (a positive hump in the medium term).

The parameters are estimated by minimizing the sum of squared errors between model rates and observed rates:

$$\hat{\beta} = \arg \min_{\beta} \sum_{i=1}^n (r_{\text{obs}}(t_i) - r(t_i; \beta))^2$$

subject to $\beta_0 > 0$ and $\lambda_1, \lambda_2 > 0$. This is a nonlinear least squares problem solved numerically via L-BFGS-B optimization in [curves.py](#).

In the notebook, we use synthetic data of spot rates to compare the three interpolation methods and its respective forward curves. In Figure 5 we can see the smoothness, discontinuity and flattening of forward rates mentioned above.

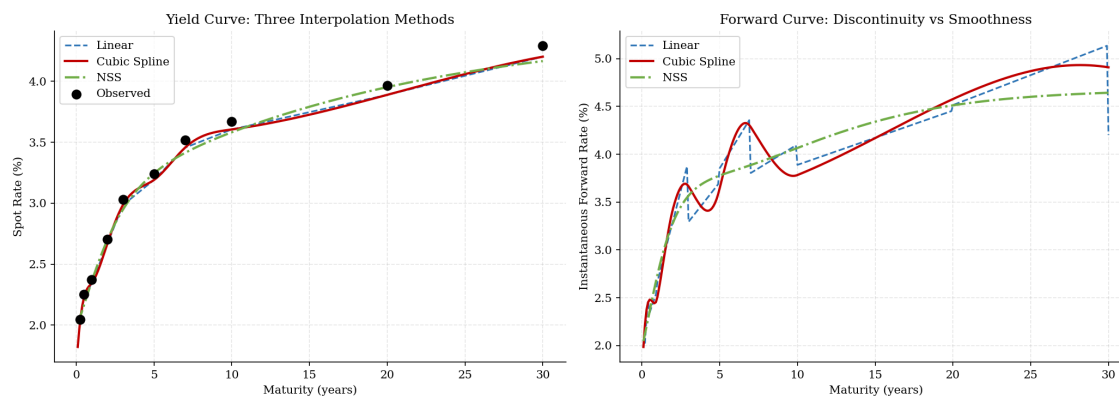


Figure 5: Three Interpolation Methods

Taking 4 key-dates (Pre-COVID, COVID shock, Fed Hikes Peak Dec-2022, and Jan-2025), we see plotted the NSS yield curve for spot and forward rates. Figure 6 shows how sloping spot curve implies that the market expects higher rates in the future.

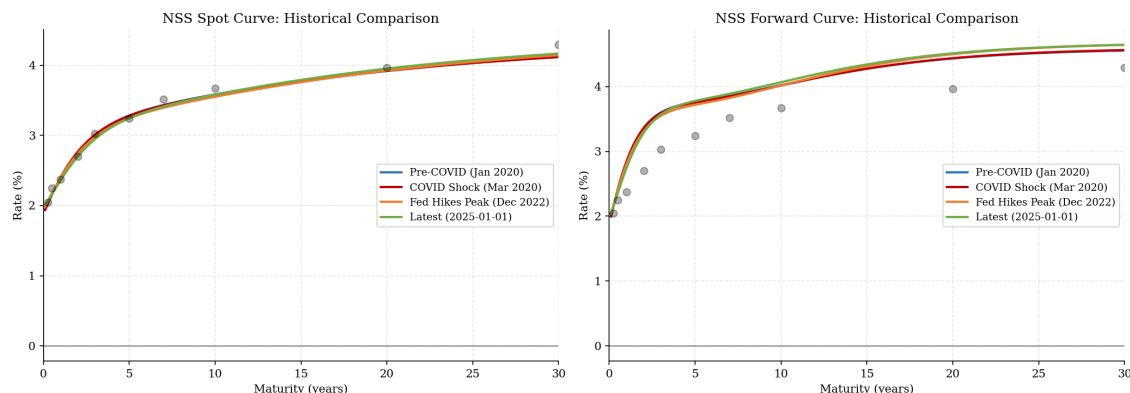


Figure 6: NSS Historical Comparison

The three Nelson-Siegel factors (level β_0 , slope β_1 and curvature β_2) are not arbitrary. [Litterman et al. \(1991\)](#) showed empirically that the three first components of the yield curve changes explains over 99% of total variance and correspond exactly to parallel shifts (level), tilts (slope) and humps (curvature). This connection between the parametric NSS model and the statistical PCA of module 4 is one of the deepest results in fixed income and provides a powerful cross-validation: a well-calibrated NSS model should produce factors that align closely with the PCA components.

2.6 Conclusions and Key Takeaways

- The discount factor $Z(t) = e^{-r(t) \cdot t}$ is the fundamental object of the yield curve: all pricing, forward rates and bootstrapping results follow from it.
- The instantaneous forward rate $f(t) = r(t) + t \cdot r'(t)$ encodes the slope of the spot curve and is continuous only when $r'(t)$ is continuous, a property guaranteed by the cubic spline but not by linear interpolation.
- NSS imposes an economic structure on the curve with six interpretable parameters: long-run level β_0 , short-run adjustment β_1 , two curvature components β_2, β_3 and two decay parameters λ_1, λ_2 . It is the standard model of central banks worldwide precisely because its factors correspond to the three dominant modes of yield curve variation.

The yield curve $r(t)$ and its discount factors $Z(t_i)$ are now complete building blocks. With them we can price any fixed-rate instrument by discounting its cash flows at maturity-specific rates rather than a single constant. However, having a precise price is only the first step. The next question is how sensitive that price is to changes in the curve: if rates rise by 5 basis points at the 5-year node, how much does a bond portfolio lose? Does the exposure come from the short end or the long end of the curve? These questions define **interest rate risk**, and answering them requires moving from a single DV01 number, which assumes a parallel shift of the entire curve, to a full vector of **Key Rate Durations** that decomposes the sensitivity node by node.

Formally, the parallel DV01 of Module 1 measured:

$$\text{DV01} = -\frac{dP}{dy} \cdot 0.0001$$

under the assumption $\frac{\partial r(t_i)}{\partial y} = 1$ for all i . Module 3 relaxes this assumption and measures instead:

$$\text{KRD}_k = -\frac{1}{P} \frac{\partial P}{\partial r(t_k)}$$

for each node t_k independently, yielding a complete picture of where interest rate risk lives along the curve.

3 Module 3: Sensitivity Analysis

Python Implementation

Module: `src/sensitivity.py`
Classes: `BondSensitivity` and `PortfolioSensitivity`
Notebook: `notebooks/03.sensitivity.ipynb`

3.1 Introduction and Class Hierarchy

Module 2 delivered a complete yield curve $r(t)$ and its discount factors $Z(t_i)$. This module asks: how sensitive is the value of a bond or portfolio to movements in that curve? Module 1 answered this with a single number, the parallel DV01, under the restrictive assumption that all rates move by the same amount. Here we relax that assumption entirely. The key insight is that interest rate risk is not scalar but **vectorial**: a portfolio has a different sensitivity at each point of the curve, and understanding that vector is what separates a risk manager who can hedge from one who cannot. The two classes in `src/sensitivity.py` implement this decomposition for a single bond and for a portfolio.

```
src/sensitivity.py
|
|-- BondSensitivity
|   |
|   |-- Inputs
|   |   |-- bond    VanillaBond instance
|   |   |-- curve   DiscountCurve instance
|   |
|   |-- Pricing
|   |   |-- _price(curve)    P = sum CF_i * Z(t_i)
|   |
|   |-- Parallel Risk
|   |   |-- dv01_parallel()
|   |   |-- modified_duration_curve()
|   |
|   |-- Key Rate Risk
|   |   |-- key_rate_durations()    KRD_k vector
|   |   |-- dv01_by_key_rate()      DV01_k vector
|   |
|   |-- Summary
|   |   |-- summary()    All metrics + additivity check
|   |
|-- PortfolioSensitivity
|   |
|   |-- Inputs
|   |   |-- bonds    List of VanillaBond
|   |   |-- curve    DiscountCurve
```

```

|         |-- weights  Units held per bond
|         |-- labels   Bond identifiers
|
|--| Aggregation
|     |-- portfolio_value()
|     |-- dv01_parallel()
|     |-- key_rate_durations()    Value-weighted KRD
|     |-- dv01_by_key_rate()     Summed DV01_k
|
|--| Applications
|     |-- scenario_pnl(curve_shift)
|     |-- hedge_ratios(hedges)
|
|--| Reporting
|     |-- summary_table()        One row per bond + total

```

The design mirrors the mathematical structure exactly. **BondSensitivity** wraps a **VanillaBond** and a **DiscountCurve** and computes sensitivities by perturbing the curve node by node. **PortfolioSensitivity** aggregate those individual vectors across bonds:

```

1  # Single bond sensitivity
2  sens = BondSensitivity(bond, curve)
3  krd = sens.key_rate_durations() # {t_k: KRD_k}
4  dv01 = sens.dv01_by_key_rate() # {t_k: DV01_k}
5  sm = sens.summary() # Additivity check included
6
7  # Portfolio aggregation
8  port = PortfolioSensitivity(
9      bonds = [bond_2y, bond_7y, bond_30y],
10     curve = curve,
11     weights = [10, 5, 2]
12 )
13 pnl = port.scenario_pnl(curve_shift) # dv ~ -DV01 * dr

```

3.2 Parallel Shift DV01

In module 1 we defined DV01 as the price sensitivity of a single bond to a 1 bp parallel shift in its yield. Here we extend this concept to the full yield curve setting, where the discount curve $r(t)$ replaces the constant yield y .

The price of a bond under the full curve is:

$$P = \sum_{i=1}^n CF_i \cdot Z(t_i) = \sum_{i=1}^n CF_i \cdot e^{-r(t_i) \cdot t_i}$$

Definition 3.1 (Parallel Shift). A parallel shift of Δy moves every point on the curve by the same amount

$$r(t) \rightarrow r(t) + \Delta y, \forall t$$

The DV01 under a parallel shift is then:

$$\text{DV01}_{\text{Parallel}} = -\frac{\partial P}{\partial y} \cdot 0.0001 \approx \frac{P(r(t) - 0.0001) - P(r(t) + 0.0001)}{2}$$

where the numerical approximation uses a central finite difference. This is identical to the single-bond DV01 of module 1 when the curve is flat, but extends naturally to arbitrary curve shapes. In terms of the Modified Duration:

$$\text{DV01}_{\text{Parallel}} = D_{\text{mod}} \cdot P \cdot 0.0001$$

where D_{mod} is computed under the full curve rather than a single yield. Figure 7 shows DV01 against maturity and the price impact of a change of 100 bps in the yield. We can see that the longer the maturity, the longer the DV01.

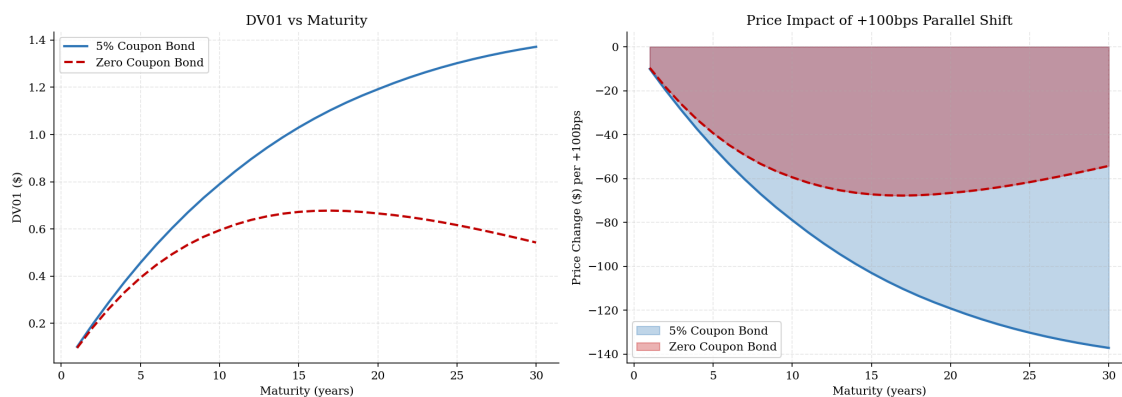


Figure 7: DV01 against Maturity and Price Impact of $\Delta y = 1\%$

The parallel DV01 captures the aggregate interest rate risk of the bond but tells us nothing about which part of the the curve drives that risk. This limitation motivates the *Key Rate Durations*.

3.3 Key Rate Durations

As we noticed before, the parallel shift assumption is a significant simplification. In practice, yield curves do not move in parallel: short rates and long rates respond differently to monetary policy, economic conditions and supply-demand dynamics. Key Rate durations (KRDs), introduced by Ho (1992), decompose the total interest rate sensitivity into contributions from each point on the curve.

Definition 3.2 (Key Rate Durations). The key rate duration at a node t_k is the price sensitivity of the bond to a 1 bp shift at that specific maturity, holding all the other rates constant:

$$\text{KRD}_k = -\frac{1}{P} \frac{\partial P}{\partial r(t_k)}$$

In practice this is computed numerically via central finite differences:

$$\text{KRD}_k \approx \frac{P(r(t_k) - \varepsilon) - P(r(t_k) + \varepsilon)}{2\varepsilon \cdot P}$$

where $\varepsilon = 0.0001$ (1 bp) and all the other rates remain fixed. The result is a vector of sensitivities, one per key rate node:

$$\mathbf{KRD} = (\text{KRD}_1, \text{KRD}_2, \dots, \text{KRD}_m)$$

KRD is not only a partial derivative, when a node changes, the movement spreads across the adjacent points: the shape of the perturbation depends of the interpolation method chosen (see for instance module 2). The KRD vector has two fundamental properties.

1. **Additivity:** The sum of all KRDs equals the Modified Duration under the full curve:

$$\sum_{k=1}^m \text{KRD}_k = D_{\text{mod}}$$

This provides a natural validation: If the KRDs do not sum to the total Modified Duration, there is an error in the implementation. Figure 8 shows four bonds (2Y 4%, 7Y 5%, 30Y 5% and 10Y 0%) and shows the KRD across the key rate nodes, also calculating the modified duration, confirming numerically the additivity property. We have to be careful, because this result is true only when there is continuous capitalization of the yield. Additivity is exact when each node of bootstrapping corresponds exactly to a cash flow, which is guaranteed under continuous capitalization. In discrete capitalization there is a small cross term that makes additivity an approximated property.

2. **Locality:** KRD_k is large when cash flows are concentrated near maturity t_k and small otherwise. A zero coupon bond maturing at t_k has $\text{KRD}_k = t_k$ and all the other KRDs equal to zero.

Proposition 3.1. *For a bond price under continuously compounded curve, $\sum_{k=1}^m \text{KRD}_k = D_{\text{mod}}$.*

Proof. A parallel shift Δy can be decomposed as the sum of individual key rate shifts of Δy at each node. Since price is linear in small shifts:

$$-\frac{1}{P} \frac{\partial P}{\partial y} = -\frac{1}{P} \sum_{k=1}^m \frac{\partial P}{\partial r(t_k)} = \sum_{k=1}^m \text{KRD}_k$$

The left-hand side is exactly D_{mod} by definition. □

The dollar sensitivity at each node is:

$$\text{DV01}_k = \text{KRD}_k \cdot P \cdot 0.0001$$

So the total DV01 satisfies:

$$\text{DV01}_{\text{Parallel}} = \sum_{k=1}^m \text{DV01}_k$$

This decomposition is directly what risk systems report to risk managers: not a single number but a vector of sensitivities showing exactly which part of the curve a position is exposed to.

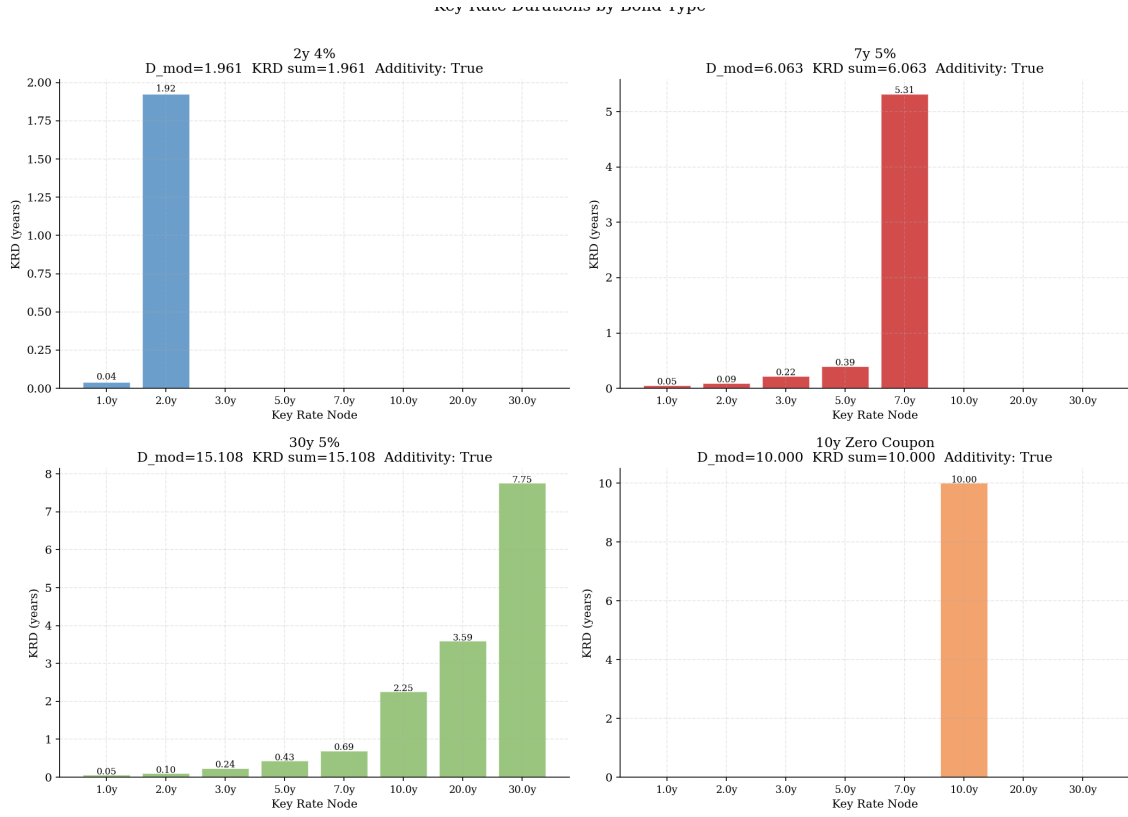


Figure 8: Key Rate Duration by Bond Type

3.4 Portfolio Sensitivity

The KRD framework extends naturally from a single bond to a portfolio. Let a portfolio consists of N bonds with prices P_j and KRD vectors $\mathbf{KRD}^{(j)}$, held in quantities w_j . The portfolio value is:

$$V = \sum_{j=1}^N w_j P_j$$

The portfolio KRD at a node t_k is the value-weighted average:

$$\text{KRD}_k^{\text{portfolio}} = \frac{\sum_{j=1}^N w_j P_j \cdot \text{KRD}_k^{(j)}}{V}$$

and the portfolio DV01 at each node:

$$\text{DV01}_k^{\text{portfolio}} = \sum_{j=1}^N w_j \cdot \text{DV01}_k^{(j)}$$

This vector representation enables two powerful risk management applications.

1. **Hedging:** Given a target portfolio with DV01 vector $\mathbf{d}^{\text{target}}$ and a set of hedging instruments with DV01 vectors $\mathbf{d}^{(j)}$, the hedge ratios h_j solves:

$$\sum_j h_j \mathbf{d}^{(j)} = -\mathbf{d}^{\text{target}}$$

This is a linear system that can be solved exactly if the number of instruments equals the number of key rate nodes, or approximately via least squares otherwise. A perfect hedge of KRDs does not only protect against parallel shifts, but against curve shifts (butterfly shifts) and slope changes (twists).

2. **Scenario Analysis:** Given a non-parallel curve shift $\Delta \mathbf{r} = (\Delta r(t_1), \dots, \Delta r(t_m))$, the approximate portfolio $P\&L$ is:

$$\Delta V \approx -V \sum_{k=1}^m \text{KRD}_k \cdot \Delta r(t_k) = -\mathbf{d}^{\text{portfolio}} \cdot \Delta \mathbf{r}$$

This is the inner product of the DV01 vector with the curve shift vector: a simple but powerful tool for stress testing.

Consider a portfolio with three bonds: 2Y 4%, 7Y 5%, 30Y 5%. Figure 9 shows the stacked

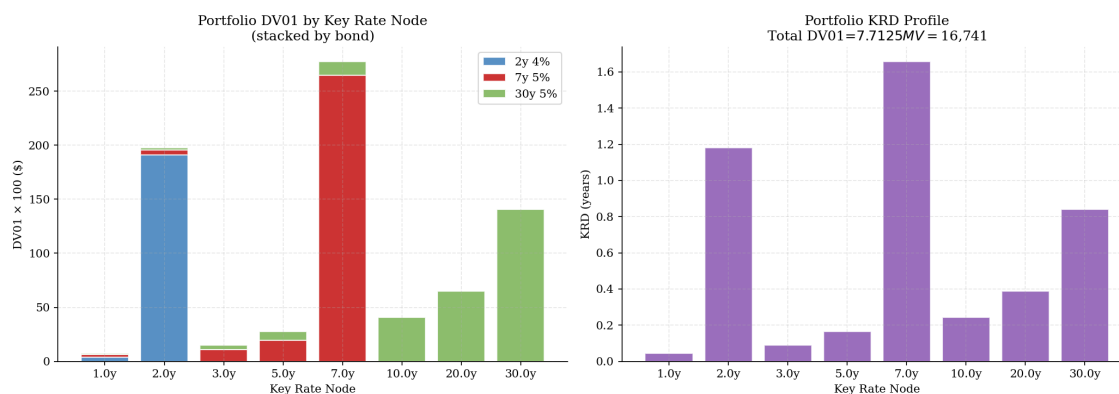


Figure 9: Stacked KRD of a 3-Bond-Portfolio

key rate duration by nodes, where the highest KRDs concentrate at the maturities of the bonds conforming the portfolio.

3.5 Conclusions and Key Takeaways

- The parallel DV01 is a useful summary statistic but hides the true structure of risk. Two portfolios with identical parallel DV01 can have very different exposures to curve steepening, flattening or butterfly moves, only the KRD vector reveals this.
- The additivity property $\sum_k \text{KRD}_k = D_{\text{mod}}$ is both a mathematical result and a practical validation tool: any implementation that violates it contains an error.

- Hedge ratios from the linear system $\sum_j h_j \mathbf{d}^{(j)} = -\mathbf{d}^{\text{target}}$ may require short positions. This is economically correct and standard in practice: a portfolio concentrated at the long end is hedged by shorting long-dated instruments, not by buying short-dated ones.

The KRD vector $\mathbf{KRD} \in \mathbb{R}^m$ has one entry per curve node (typically 10 or more). Working with this full vector is informative but computationally expensive and statistically redundant: the nodes are highly correlated, and most of the movement in the yield curve is driven by a small number of underlying forces. A natural question is: can we find a lower-dimensional representation that captures most of the risk without losing important information? The answer is yes, and it comes from the covariance structure of historical yield curve changes. If $\mathbf{\Sigma}$ is the covariance matrix of daily rate changes, then the portfolio P&L variance is:

$$\text{Var}(\Delta V) = \mathbf{d}^T \mathbf{\Sigma} \mathbf{d}$$

where $\mathbf{d} \in \mathbb{R}^m$ is the DV01 vector of this module. Evaluating this expression requires estimating and inverting $\mathbf{\Sigma}$, a full $m \times m$ matrix. Module 4 decomposes $\mathbf{\Sigma}$ via PCA into three dominant eigenvectors: *level*, *slope* and *curvature* and shows that:

$$\text{Var}(\Delta V) \approx \sum_{k=1}^3 \lambda_k (\mathbf{v}_k^T \mathbf{d})^2$$

reducing the problem from a full $m \times m$ matrix to three scalar projections. The DV01 vector $\mathbf{d}$ computed here is the direct input to that calculation.

4 Module 4: PCA on Term Structure

Python Implementation

Module: `src/pca.py`
Classes: `YieldCurvePCA` and `PCARiskManager`
Notebook: `notebooks/04_pca.ipynb`

4.1 Introduction and Class Hierarchy

Modules 2 and 3 together produce a yield curve $r(t)$ and a KRD DV01 vector $\mathbf{d} \in \mathbb{R}^m$ that describes where risk lives along the curve. The natural next question is: can we compress that m -dimensional risk vector into something smaller without losing important information? The answer exploits a remarkable empirical regularity. Daily changes in yield curves across all major markets are highly correlated: when rates move, they rarely move in arbitrary directions. Instead they move along a small number of dominant patterns (level, slope and curvature) that account for over 99% of total variance. PCA discovers these patterns statistically from historical data and transforms the $m \times m$ covariance problem of Module 3 into three scalar projections. The two classes in `src/pca.py` implement this decomposition and connect it back to the risk vector $\mathbf{d}$ from Module 3.

```
src/pca.py
|
|-- YieldCurvePCA
|   |
|   |-- Inputs
|   |   |-- df_yields    Historical yield panel
|   |   |-- n_factors    Components to retain
|   |
|   |-- Core (_fit)
|   |   |-- X            Daily changes matrix
|   |   |-- Sigma        Covariance matrix
|   |   |-- eigenvalues  lambda_1 >= ... >= lambda_m
|   |   |-- eigenvectors v_1, ..., v_m
|   |
|   |-- Variance
|   |   |-- variance_explained(K)  R2_K
|   |   |-- variance_table()       Full table
|   |
|   |-- Factor Structure
|   |   |-- loadings(K)             V_K matrix
|   |   |-- scores(K)              F = X @ V_K
|   |   |-- factor_shock(k, sigma) Stress scenario
|   |
|   |-- Reconstruction
|   |   |-- reconstruct(K)          X_hat = F_K @ V_K.T
|   |   |-- reconstruction_error(K) MSE analytical
```

```

|      |
|      |-- NSS Alignment
|      |      |-- nss_alignment(lambda)  R2 per factor
|-- PCARiskManager
|
|      |-- Inputs
|      |      |-- pca          YieldCurvePCA instance
|      |      |-- dv01_vec    KRD DV01 vector from M3
|
|      |-- Factor Risk
|      |      |-- factor_dv01()      DV01_PCk = v_k.T @ d
|      |      |-- total_dv01()      Sum of KRD DV01s
|      |      |-- pnl_variance(K)    Var(dV) exact + approx
|
|      |-- Immunization
|      |      |-- immunization_check()  DV01_PC1 = 0?
|
|      |-- Scenarios
|      |      |-- scenario_pnl(shocks)
|      |      |-- summary()

```

The two classes are designed to be used in tandem. `YieldCurvePCA` is fitted once on historical data and exposes the eigenstructure of the covariance matrix. `PCARiskManager` takes a fitted PCA object and a DV01 vector from Module 3 and translates node-level risk into factor-level risk:

```

1 # Fit PCA on historical yields
2 pca = YieldCurvePCA(df_yields, n_factors = 3)
3
4 # Connect to portfolio DV01 from Module 3
5 rm = PCARiskManager(pca, dv01_vec)
6
7 # Factor DV01 decomposition
8 fdv01 = rm.factor_dv01()
9 # {'PC1 Level': 0.082, 'PC2 Slope': -0.011, ...}
10
11 # P&L variance under factor model
12 pnl = rm.pnl_variance()
13
14 # Immunization check: is DV01_PC1 ~ 0?
15 imm = rm.immunization_check()

```

4.2 Motivation

The yield curve does not move randomly: when the FED raises rates, the entire curve tends to shift upward. When the economy slows, short rates fall faster than long rates, steepening the curve. These patterns suggest that the high-dimensional object we call the yield curve (a vector of rates at 10 or more maturities) is driven by a small number of underlying factors. Principal Component Analysis (PCA) formalizes this intuition. Given a historical panel of yield curve changes, PCA finds the directions of maximum variance in the data: the directions along which the curve moves

the most. Litterman et al. (1991) showed empirically that three such directions explains over 99% of total yield curve variance, and that they correspond exactly to parallel shifts (level), tilts (slope) and humps (curvature). This result has profound implications both for risk management and for the connection with the NSS model of module 2. To motivate this, notice how different maturities correlate. Figure 10 shows how maturities co-move and that they are highly correlated

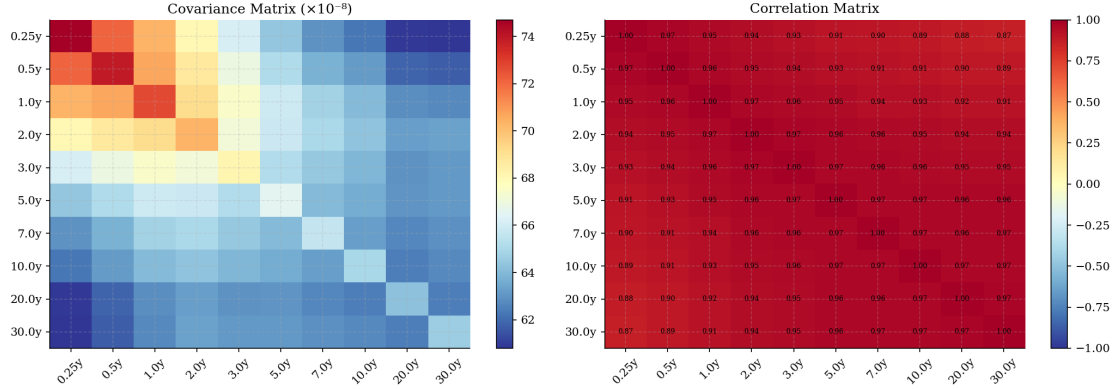


Figure 10: Covariance and Correlation Matrix

at neighboring maturities and less so at the extremes.

4.3 PCA Framework

Let $\Delta \mathbf{r}_t \in \mathbb{R}^m$ be the vector of daily changes in spot rates at m maturities on day t :

$$\Delta \mathbf{r}_t = (\Delta r(t_1), \Delta r(t_2), \dots, \Delta r(t_m))^T$$

Over T trading days we observe the data matrix:

$$\mathbf{X} \in \mathbb{R}^{T \times m}$$

where row t is the vector of rate changes on day t . After centering (subtracting the column means), the empirical covariance matrix is:

$$\Sigma = \frac{1}{T-1} \mathbf{X}^T \mathbf{X} \in \mathbb{R}^{m \times m}$$

Definition 4.1 (Principal Components). The principal components of $\mathbf{X}$ are the eigenvectors of Σ , obtained via the spectral decomposition:

$$\Sigma = \mathbf{V} \Lambda \mathbf{V}^T$$

where $\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_m)$ with $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_m \geq 0$ are the eigenvalues and $\mathbf{V} = [\mathbf{v}_1 | \mathbf{v}_2 | \dots | \mathbf{v}_m]$ is the orthogonal matrix of eigenvectors, so that $\mathbf{V}^T \mathbf{V} = \mathbf{I}$.

Each eigenvector $\mathbf{v}_k \in \mathbb{R}^m$ is a *factor loading vector*: it describes a pattern of simultaneous rate movements across all maturities. The corresponding eigenvalue λ_k equals the variance explained by that factor.

Definition 4.2 (Factor Scores). The projection of the data onto the k -th principal component is the factor score:

$$\mathbf{f}_k = \mathbf{X}\mathbf{v}_k \in \mathbb{R}^T$$

It represents the time series of how much each daily curve move was driven by factor k .

The total variance in the data is the trace of the covariance matrix:

$$\text{Total Variance} = \text{tr}(\Sigma) = \sum_{k=1}^m \lambda_k$$

The fraction of variance explained by the first K principal components is:

$$R_K^2 = \frac{\sum_{k=1}^K \lambda_k}{\sum_{k=1}^m \lambda_k}$$

In practice for U.S. Treasury yields:

$$R_1^2 \approx 80\% \quad R_2^2 \approx 95\% \quad R_3^2 \approx 99\%$$

This means that three numbers (the scores on three factors) capture essentially all the information in the daily movement of ten or more yield curve points. The remaining factors are noise. Using synthetic data in the notebook, we found the following results.

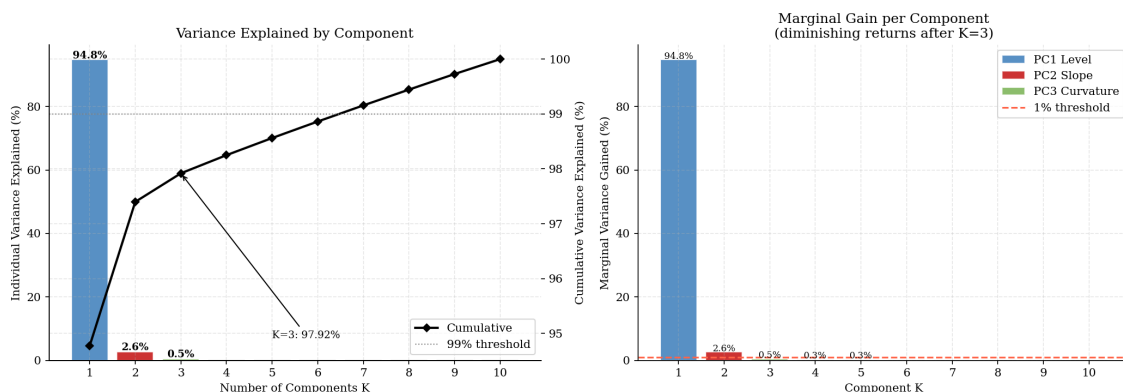


Figure 11: Variance Explained by Component

Figure 11 shows the explained variance across components, and it is clear that a $\approx 98\%$ of the variance is explained by the first 3 components.

4.4 The Three Factors (Level, Slope and Curvature)

The empirical result of [Litterman et al. \(1991\)](#) is that the three dominant eigenvectors have stable and economically interpretable shapes across markets and time periods:

1. **PC1 (Level):** The first eigenvector $\mathbf{v}_1$ has entries of approximately equal sign and magnitude across all maturities. A unit move along PC1 shifts the entire curve up or down by roughly the same amount at every maturity. This is, the parallel shift scenario of module 3.

$$v_{1,j} \approx \frac{1}{\sqrt{m}}, \forall j$$

2. **PC2 (Slope):** The second eigenvector $\mathbf{v}_2$ has positive entries at short maturities and negative entries at long maturities (or vice versa depending on sign convention). A unit move along PC2 steepens or flattens the curve: short rates move in the opposite direction to long rates.
3. **PC3 (Curvature):** The third eigenvector $\mathbf{v}_3$ has entries that are positive at short and long maturities and negative at intermediate maturities (or vice versa). A unit move along PC3 creates a hump in the middle of the curve, corresponding to the curvature parameter β_2 in the NSS model.

This connection with NSS is not coincidental. Both frameworks are describing the same underlying structure: the yield curve lives in a low-dimensional subspace spanned by level, slope and curvature. NSS imposes this structure parametrically from the start; PCA discovers it statistically from the data.

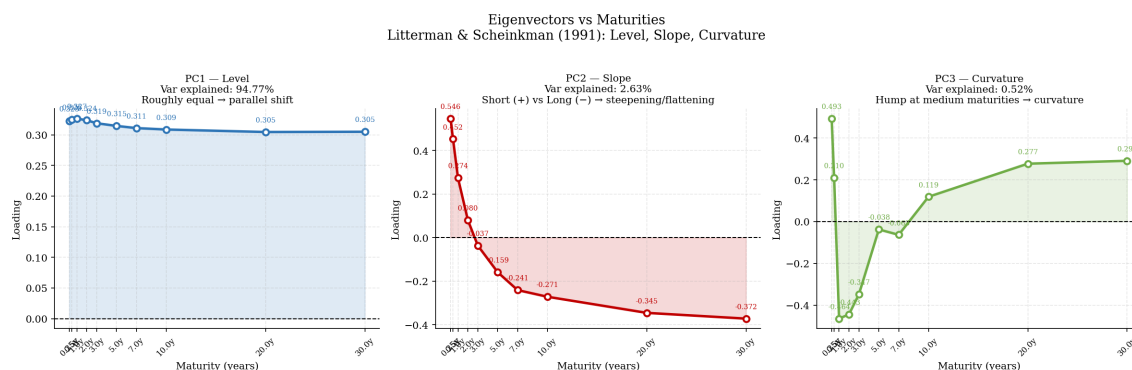


Figure 12: Eigenvectors vs Maturities

We can visualize the eigenvectors associated with the first three components in Figure 12. We can easily see what we stated, the first eigenvector $\mathbf{v}_1$ (level) is relatively flat across all maturities (parallel shift), the second eigenvector $\mathbf{v}_2$ (slope) has positive entries at short and negative entries at long maturities. The third eigenvector $\mathbf{v}_3$ (curvature) has positive entries in the very short maturities, negative entries at medium maturities and positive again at long maturities.

Figure 13 shows the time series of each factor score $f_{k,t} = \mathbf{X}_t \cdot \mathbf{v}_k$ over the sample period. Three observations stand out. First, the PC1 score has the largest amplitude by construction: its variance equals λ_1 , the largest eigenvalue, so it is the most volatile series. Second, PC2 and PC3 scores are progressively quieter, reflecting their smaller eigenvalues. Third, the scores carry economic content: sustained negative values of the PC2 score correspond to periods of yield curve inversion (short rates rising relative to long rates), while sharp spikes in the PC3 score correspond to episodes where the belly of the curve moved independently of the wings, typical during central bank forward guidance transitions.

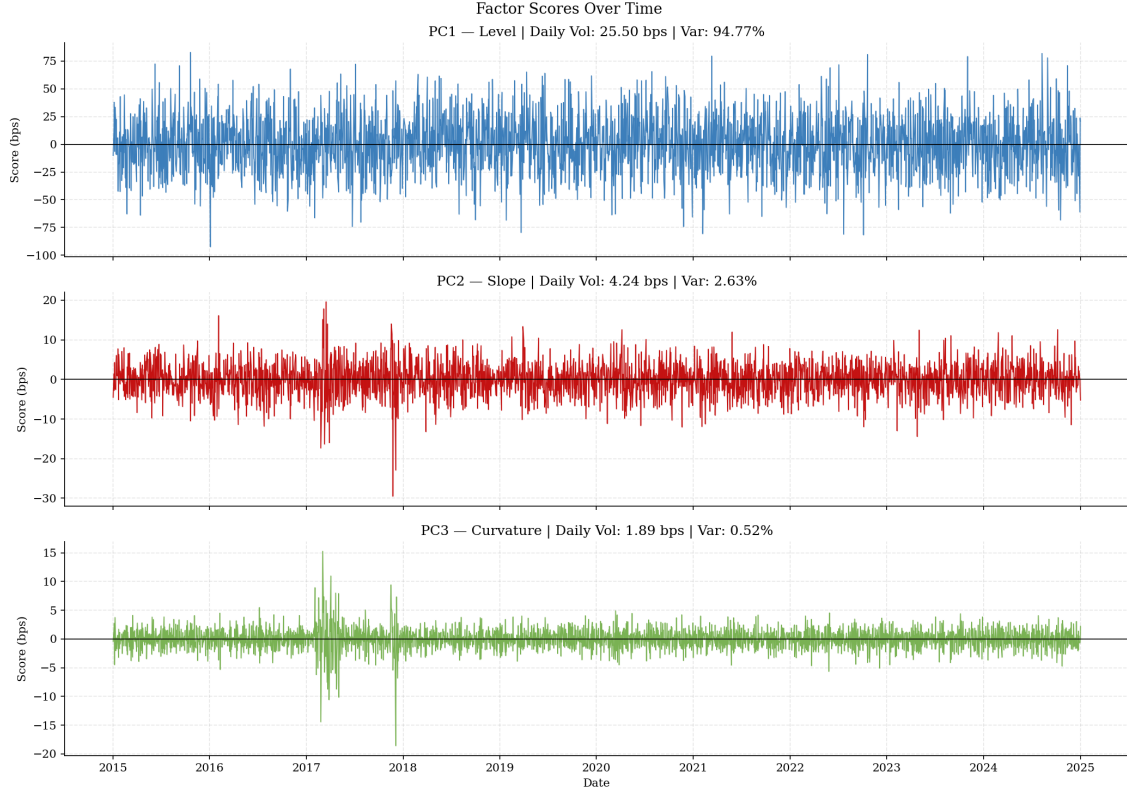


Figure 13: Factor Scores over Time

4.5 PCA Reconstruction and Risk Management

Given the first K principal components, the yield curve change on day t can be approximated as

$$\Delta \mathbf{r}_t \approx \sum_{k=1}^K f_{k,t} \cdot \mathbf{v}_k = \mathbf{V}_K \mathbf{f}_{K,t}$$

where $\mathbf{V}_K \in \mathbb{R}^{m \times K}$ contains the first K eigenvectors and $\mathbf{f}_{K,t} \in \mathbb{R}^K$ are the factor scores on day t . The reconstruction error is:

$$\varepsilon_t = \Delta \mathbf{r}_t - \mathbf{V}_K \mathbf{f}_{K,t}$$

and its mean squared error across all days and maturities is exactly:

$$\text{MSE} = \frac{1}{T} \sum_{t=1}^T \|\varepsilon_t\|^2 = \sum_{k=K+1}^m \lambda_k$$

The sum of the discarded eigenvalues. This provides a clean analytical formula for the reconstruction error without having to compute it empirically. A powerful application of the reconstruction is the

definition of stress scenarios. Instead of moving nodes randomly, we can move the factors x standard deviations and see what happens, generating a plausible market scenario. This reconstruction error

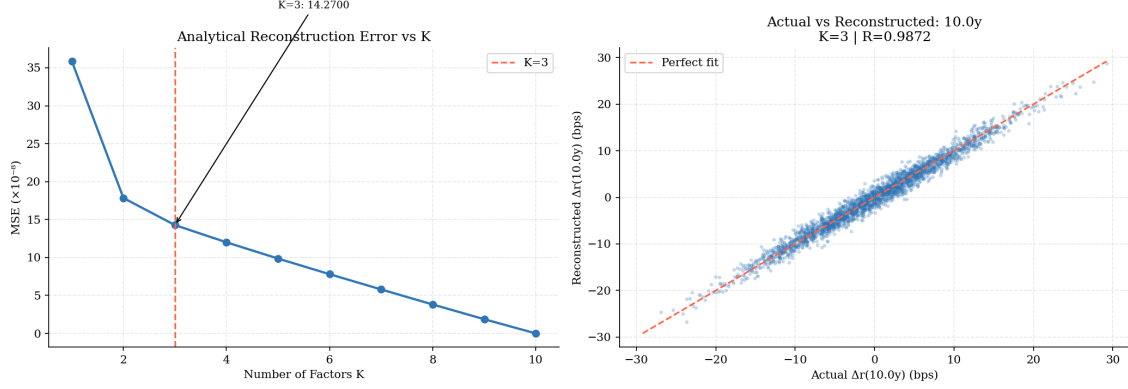


Figure 14: PCA Reconstruction

is visualized in Figure 14, where we can see the clear match between the reconstructed yield curve and the actual $\Delta \mathbf{r}_t$.

The PCA decomposition transforms the problem of managing risk at m correlated nodes into the problem of managing risk along K uncorrelated factors. The DV01 of the bond portfolio along factor k is:

$$\text{DV01}_{PC_k} = \mathbf{v}_k^T \cdot \mathbf{d}$$

where $\mathbf{d} \in \mathbb{R}^m$ is the vector of key rate DV01 from module 3. Since the factors are orthogonal, the total variance of the portfolio P&L is:

$$\text{Var}(\Delta V) = \sum_{k=1}^K (\text{DV01}_{PC_k})^2 \lambda_k$$

This is much simpler than working with the full $m \times m$ covariance matrix of key rate sensitivities, and is exactly the computation performed by risk engines under a factor risk model. If we want for instance a *level neutral* portfolio, we search for $\text{DV01}_{PC_1} = 0$, this is more robust than traditional immunization by duration because PCA knows that the curve does not change exactly parallel. This is known as **Factor Immunization**.

Proposition 4.1. *Under the factor model $\Delta \mathbf{r} = \sum_{k=1}^K f_k \mathbf{v}_k$ with $\text{Var}(f_k) = \lambda_k$ and uncorrelated factors, the portfolio P&L variance is:*

$$\text{Var}(\Delta V) = \mathbf{d}^T \Sigma \mathbf{d} \approx \sum_{k=1}^K (\mathbf{v}_k^T \mathbf{d})^2 \lambda_k$$

Proof. The P&L is $\Delta V = -\mathbf{d}^T \Delta \mathbf{r}$. Its variance is:

$$\text{Var}(\Delta V) = \mathbf{d}^T \Sigma \mathbf{d} = \sum_{k=1}^m (\mathbf{v}_k^T \mathbf{d})^2 \lambda_k$$

Truncating to the first K factors gives the approximation. □

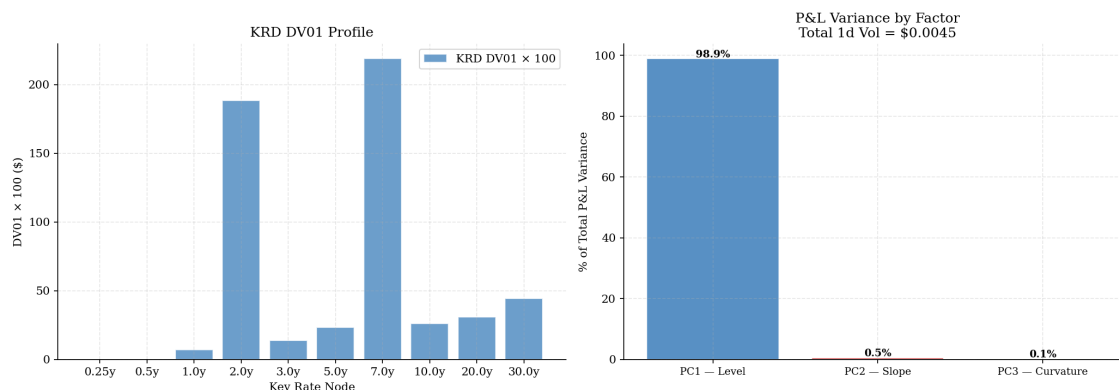


Figure 15: Factor Risk Management

For a portfolio with three bonds (2Y 4%, 7Y 5%, 30Y 5%), Figure 15 plots $DV01_{PC_k}$ against KRDs, and the total P&L variance explained by each component.

The connection between PCA and NSS runs deeper than a superficial analogy. Define the NSS loading vectors:

$$\phi_0 = 1 \quad \phi_1(j) = \frac{1 - e^{-t_j/\lambda}}{t_j/\lambda} \quad \phi_2(j) = \frac{1 - e^{-t_j/\lambda}}{t_j/\lambda} - e^{-t_j/\lambda}$$

evaluated at each maturity t_j . These are the columns of the NSS design matrix. The claim is that the empirical eigenvectors $\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3$ from PCA are well approximated by ϕ_0, ϕ_1, ϕ_2 up to normalization and sign. This can be verified empirically: regress each PCA eigenvector onto the NSS loading vectors and compute the R^2 . In practice, $R^2 = 0.95$ for all three factors, confirming that NSS and PCA are discovering the same underlying structure from different angles: one parametric, one statistical. If R^2 is low, it means that λ in the NSS model is not well-calibrated to capture the current historic dynamic of the market. PCA factors are data-dependent while NSS is a fixed structure, we need to choose between real statistic or an imposed functional form. In the notebook, we see that factor 1 $\mathbf{v}_1$ is well-aligned between PCA and NSS, not the same result yields from the other factors. Figure 16 compares both eigenvectors against maturities and it is easy to see that the curves with NSS are excessively smooth. Formalizing the visualization, we have to notice that NSS has a fixed exponential functional form parametrized by λ_1 and λ_2 , while PCA captures the exact empiric form of the historic curve. The excess of smoothness of NSS in PC2 and PC3 is consequence that one and only one exponential cannot reproduce complex humps that appear in the data.

4.6 Conclusions and Key Takeaways

- Three factors explain over 99% of daily yield curve variance. The remaining components are idiosyncratic noise that cannot be hedged systematically and should not drive portfolio positioning.

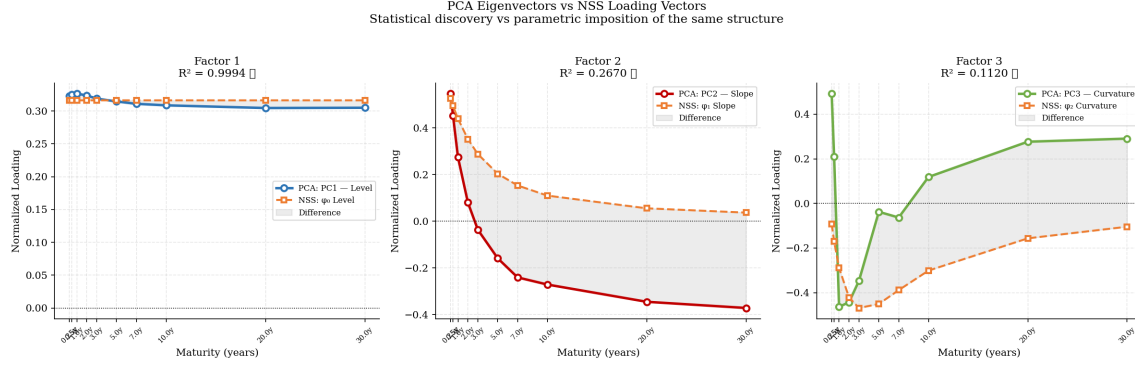


Figure 16: PCA Eigenvectors vs NSS Loading Vectors

- Factor immunization ($DV01_{PC_1} = 0$) is strictly more robust than duration matching: it neutralizes the dominant source of yield curve risk without assuming a perfectly parallel shift, which never occurs in practice.
- PCA and NSS discover the same underlying structure from different angles: one statistical, one parametric. When R^2 between eigenvectors and NSS loadings is high (> 0.90), the two models are consistent and the NSS calibration can be trusted. When R^2 is low for PC2 or PC3, it signals that the chosen decay parameter λ is not well-suited to the current market regime.

The PCA of this module has a direct financial application beyond risk management. The yield curve $r(t)$ from Module 2 and its discount factors $Z(t_i)$ are the inputs to pricing the most liquid fixed income derivative in the world: the Interest Rate Swap (IRS). An IRS exchanges a fixed rate K for a floating rate (SOFR or EURIBOR) on a notional N over a sequence of dates $t_1, \dots, t_n$. Its fair fixed rate, the swap rate K^* , is determined entirely by the discount factors of Module 2:

$$K^* = \frac{1 - Z(t_n)}{\sum_{i=1}^n \delta_i \cdot Z(t_i)}$$

The connection with PCA is immediate: the DV01 of a swap is also a vector $\mathbf{d} \in \mathbb{R}^m$, concentrated at the payment dates of the fixed leg, and the factor DV01 decomposition $DV01_{PC_k} = \mathbf{v}_k^\top \mathbf{d}$ applies without modification. A swap desk that manages its book in factor space (neutralizing PC1, PC2 and PC3 exposures) is running exactly the risk framework developed across Modules 3 and 4.

5 Module 5: IRS Pricing

Python Implementation

Module: `src/swaps.py`
Classes: `VanillaIRS` and `SwapPortfolio`
Notebook: `notebooks/05_irs.ipynb`

5.1 Introduction and Class Hierarchy

The yield curve of Module 2 and the sensitivity framework of Module 3 converge in this final module to price the most liquid fixed income derivative in the world: the Interest Rate Swap (IRS). With over \$400 trillion in outstanding notional, the IRS market dwarfs all bond markets combined, and it is the instrument for which systems like Murex MX.3 were fundamentally designed. The key insight of this module is that an IRS is not a new mathematical object: it is a portfolio of Forward Rate Agreements, each of which is priced using exactly the forward rates and discount factors of Module 2. The swap rate K^* is determined entirely by the discount curve, and the swap DV01 is a vector in $\mathbb{R}^m$ that feeds directly into the PCA risk framework of Module 4. The project thus closes a complete cycle: from a single bond price to a factor-based risk decomposition of a swap portfolio.

```
src/swaps.py
|
|-- VanillaIRS
|   |
|   |-- Attributes
|   |   |
|   |   |-- N          Notional
|   |   |-- K          Fixed rate
|   |   |-- tenor      Maturity in years
|   |   |-- freq       Payment frequency
|   |   |-- curve      DiscountCurve (Module 2)
|   |   |-- position   'long' or 'short'
|   |   |-- pay_dates  Payment schedule
|   |   |-- Z          Discount factors at pay_dates
|   |
|   |-- Leg Pricing
|   |   |
|   |   |-- fixed_leg_value(curve)
|   |   |-- floating_leg_value(curve) = N always
|   |
|   |-- Swap Pricing
|   |   |
|   |   |-- swap_rate()      K* formula
|   |   |-- value(curve)    MtM = float - fixed
|   |   |-- inception_value() Validation: must be 0
|   |
|   |-- FRA Decomposition
|   |   |
|   |   |-- fra_values()    One row per period
|   |                       sum = swap value at K*
```

```

|
|
|      |-- Sensitivity
|      |      |-- dv01()           Parallel shift
|      |      |-- key_rate_dv01()  KRD DV01 vector
|      |      |-- modified_duration()
|      |      |-- value_vs_rate()  P&L profile
|      |
|      |-- Summary
|      |      |-- summary()        All metrics
|
|-- SwapPortfolio
|
|      |-- Aggregation
|      |      |-- total_value(curve)
|      |      |-- total_dv01()
|      |      |-- key_rate_dv01()
|      |
|      |-- Applications
|      |      |-- scenario_pnl(curve_shift)
|      |      |-- summary_table()
|      |
|      |-- Factory
|      |      |-- curve_trade(N, short, long, freq, curve)

```

The two classes mirror the mathematical structure exactly. `VanillaIRS` prices a single swap and exposes its complete sensitivity profile, while `SwapPortfolio` aggregates across multiple contracts and implements the classic curve trade:

```

1  # Price a 10y semi-annual swap at fair rate
2  swap = VanillaIRS(1_000_000, 0.0, 10, 2, curve)
3  K_star = swap.swap_rate()
4  swap = VanillaIRS(1_000_000, K_star, 10, 2, curve)
5
6  # Validate: inception value must be zero
7  assert abs(swap.inception_value()) < 1e-4
8
9  # FRA decomposition: sum must be zero at K = K*
10 fra = swap.fra_values()
11 assert abs(fra["FRA Value"].sum()) < 1e-4
12
13 # Curve trade: pay fixed 2y, receive fixed 10y
14 trade = SwapPortfolio.curve_trade(
15     1_000_000, 2, 10, 2, curve
16 )

```

5.2 Interest Rate Swap Definition and Structure

Definition 5.1 (Interest Rate Swap). An interest rate swap (IRS) is a contract between two counter-parties that agree to exchange cash flows at predetermined future dates $t_1, t_2, \dots, t_n$. In the most common form, the *plain vanilla IRS*, one party pays a fixed rate K and receives a floating

rate (typically SOFR or EURIBOR) ¹, both applied to a notional amount N . The net cash flow at each period i is:

$$V_i = N \cdot (L_{i-1} - K) \cdot \delta_i$$

where L_{i-1} is the floating rate observed at the start of the period i and $\delta_i = t_i - t_{i-1}$ is the day count fraction. The party paying fixed and receiving floating has a **long position**; the party paying floating and receiving fixed has a **short position**.

The fundamental insight is that a Swap is nothing more than a portfolio of forward rate agreements (FRAs), one for each payment date. Each FRA exchanges a fixed payment against a floating payment at a single future date, and summing them yields the full swap. This connection, developed in section 5.4, is the most elegant result of the module.

5.3 The Floating Leg equals Par

Before pricing the swap, we need one foundational result: the present value of the floating leg always equals the notional N at the start of each reset period. This is the key to the entire pricing framework.

Proposition 5.1 (Floating Bond = Par). *The present value of the floating leg at $t = 0$ is:*

$$V_{float} = N$$

Proof. The floating leg pays $N \cdot L_{i-1} \cdot \delta_i$ at each t_i . By the definition of the LIBOR/SOFR rate, each payments satisfies:

$$N \cdot L_{i-1} \cdot \delta_i \cdot Z(t_i) = N \cdot (Z(t_{i-1}) - Z(t_i))$$

This follows because L_{i-1} is defined precisely as the rate that makes the present value of a unit investment grow from $Z(t_{i-1})$ to $Z(t_i)$ over the period. Summing telescopically over all periods and adding the present value of the return of the notional at t_n :

$$V_{float} = N \sum_{i=1}^n (Z(t_{i-1}) - Z(t_i)) + N \cdot Z(t_n) = N(Z(t_0) - Z(t_n)) + N \cdot Z(t_n) = N \cdot Z(0) = N$$

□

The telescopic sum collapses beautifully: the floating leg is worth par regardless of the shape of the yield curve, because the floating rate resets continuously to the market rate.

5.4 Swap as a Portfolio of FRAs

Definition 5.2 (Forward Rate Agreement). A FRA is a contract where a fixed rate K is agreed today for a loan over a future period $[t_{i-1}, t_i]$. Its value at $t = 0$ is:

$$V_{FRA} = N \cdot \delta_i \cdot (f(t_{i-1}, t_i) - K) \cdot Z(t_i)$$

where $f(t_{i-1}, t_i)$ is the forward rate defined in section 2.3.

¹In modern post-crisis markets, a **multi-curve framework** is used: the floating leg is projected using a forwarding curve (e.g., EURIBOR 6 M), while all cash flows are discounted using a risk-free OIS curve (e.g., SOFR or ESTR). For simplicity, this module assumes a single-curve environment where $Z(t)$ is used for both projection and discounting.

The IRS is then exactly the sum of these FRAs:

$$V_{\text{swap}} = \sum_{i=1}^n V_{\text{FRA}}(t_i) = N \cdot \sum_{i=1}^n \delta_i \cdot (f(t_{i-1}, t_i) - K) \cdot Z(t_i)$$

This decomposition makes the pricing completely transparent: the swap is worth the present value of the difference between the forward rates the market expects and the fixed rate K the counterparty pays.

5.5 Swap Rate K^*

Definition 5.3 (Swap Rate K^*). The swap rate K^* is the fixed rate that makes the present value of the swap zero at inception:

$$V_{\text{swap}}(K^*) = 0$$

Proposition 5.2. *The swap rate is given by:*

$$K^* = \frac{1 - Z(t_n)}{\sum_{i=1}^n \delta_i \cdot Z(t_i)}$$

Proof. The swap can be decomposed as the difference between a floating bond and a fixed coupon bond:

$$V_{\text{swap}} = V_{\text{float}} - V_{\text{fixed}} = N - N \left(K \cdot \sum_{i=1}^n \delta_i \cdot Z(t_i) + Z(t_n) \right)$$

setting $V_{\text{swap}} = 0$ and dividing by N :

$$1 = K^* \cdot \sum_{i=1}^n \delta_i \cdot Z(t_i) + Z(t_n)$$

$$K^* = \frac{1 - Z(t_n)}{\sum_{i=1}^n \delta_i \cdot Z(t_i)}$$

□

The numerator $1 - Z(t_n)$ is the present value of a basis point (almost): it measures how much the discount factor has decayed from today to maturity. The denominator is the *present value of an annuity* of δ_i at each payment date and it represents the sensitivity of the swap value to a 1 bp change in the swap rate. The swap is therefore the rate of annuity whose present value matches the decay of the discount factor to maturity. In the notebook, we use the following vanilla Swap: Notional $N = 1000000$, fair swap rate $K^* = 5.4939\%$, tenor = 10Y, frequency = 2, position = long. Making a FRA decomposition, we get the FRA values by period showed in Figure 17 and validate that the sum of FRAs has to be zero.

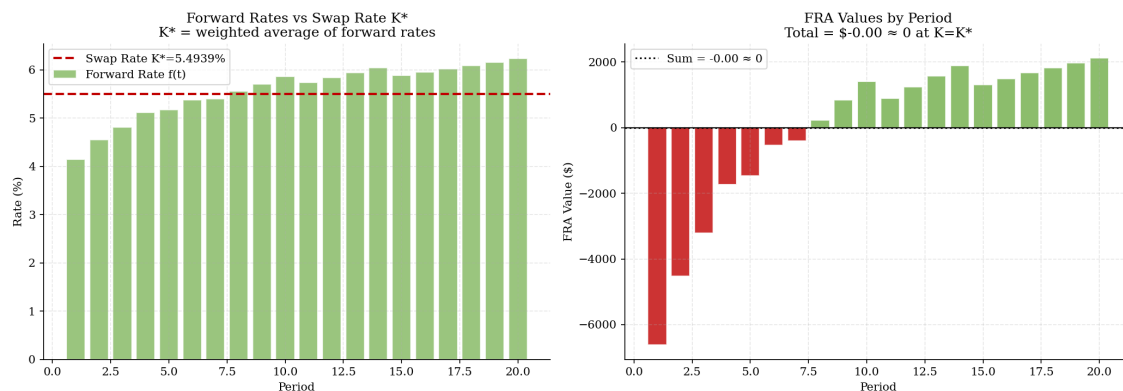


Figure 17: FRA Decomposition

5.6 Swap Value at t

Once the swap is agreed at rate K , if rates change the swap acquires positive or negative value. For the fixed payer (long position), the value at time t is:

$$V_t = V_{\text{float}}(t) - V_{\text{fixed}}(t)$$

Using the result that the floating leg always resets to par at each reset date, and recomputing the fixed leg under the new discount curve:

$$V_t = N \left[1 - Z_t(t_n) - K \sum_{i:t_i > t} \delta_i \cdot Z_t(t_i) \right]$$

where $Z_t(\cdot)$ denotes discount factors under the curve at time t . Equivalently this can be written as:

$$V_t = (K_t^* - K) \sum_{i:t_i > t} \delta_i \cdot Z_t(t_i)$$

where K_t^* is the current swap rate for the remaining tenor. If rates have risen ($K_t^* > K_t$), the fixed payer benefits because they are playing below-market fixed rate. If rates have fallen ($K_t^* < K_t$) the fixed payer loses. In Figure 18 we can visualize how the swap curve behaves against the spot and forward curve, noting that the spot curve $r(t)$ is above the swap curve $K^*(t)$ for long tenors and very similar for short tenors.

5.7 Swap DV01 and Applications

The DV01 swap under a parallel shift of the yield curve is:

$$\text{DV01}_{\text{swap}} = \text{DV01}_{\text{fixed leg}} - \text{DV01}_{\text{floating leg}}$$

Since the floating leg has very low duration (it resets to par at every payment date, so its effective duration is approximately $\delta_1/2$, half of the first period), the swap DV01 is dominated by the fixed

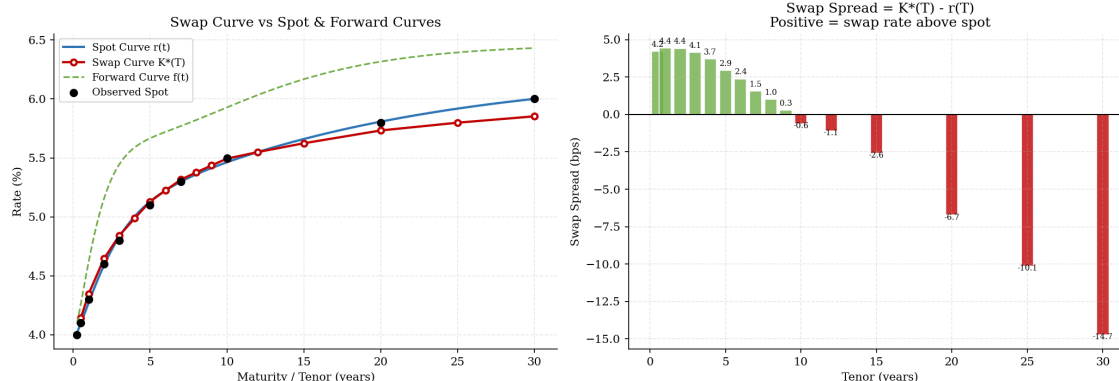


Figure 18: Swap Curve vs Spot Curve

leg:

$$DV01_{\text{swap}} \approx D_{\text{mod}}^{\text{fixed}} \cdot V_{\text{fixed}} \cdot 0.0001$$

The key rate DV01 profile of the swap is concentrated at intermediate maturities, peaking near the midpoint of the swap tenor. This contrasts sharply with a zero coupon bond, which concentrates all risk at maturity, and makes swaps particularly useful instruments for targeting specific parts of the yield curve. The swap has no convexity from the floating leg, all convexity comes from the fixed leg only. In figure 19 we can visualize a swap P&L profile against parallel shifts of the yield

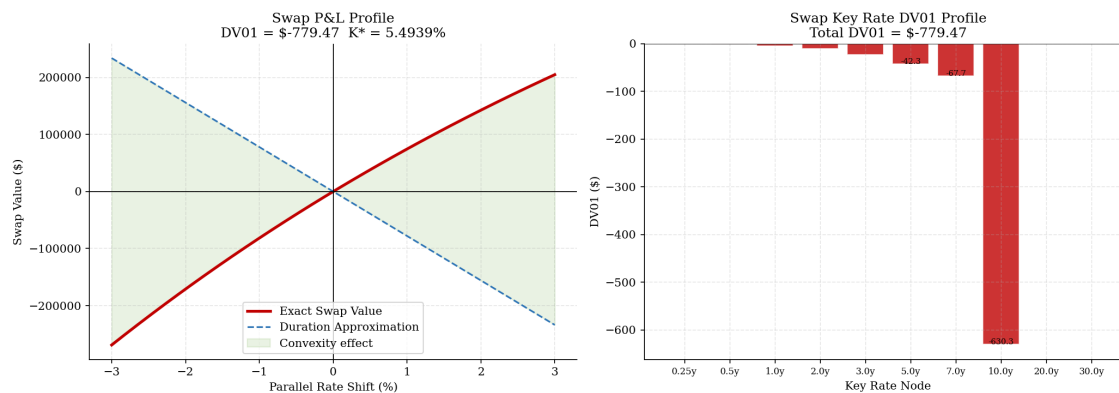


Figure 19: Swap Sensitivity Decomposition

and KRD of a DV01. We can notice that the key rate nodes gets more relevant at long tenors near to maturity and that positive changes in level of the yield increases the value of the long position swap and when rates rise, the fixed payer benefits because they locked in a below-market fixed rate. Swaps are the most liquid fixed income derivatives in the world, with outstanding notional exceeding \$400 trillion. Their three main uses are:

1. **Debt transformation:** A corporate issuer with floating rate debt (paying LIBOR + spread) can enter a swap paying fixed and receiving floating, effectively converting the floating obligation to a fixed rate. The swap eliminates the uncertainty of future interest payments.
2. **Duration management:** A pension fund or insurance company with long-dated liabilities wants to increase the duration of its asset portfolio without buying bonds. By receiving fixed in a long-dated IRS, the fund increases duration efficiently and at low transaction cost, since the swap requires no upfront payment.
3. **Yield curve positioning:** A portfolio manager who believes the yield curve will flatten can receive fixed in a long-dated swap and pay fixed in a short-dated swap simultaneously, creating a position that profits from flattening regardless of the absolute level of rates. This is the classic **curve trade** in fixed income.
4. **Economic Advantage:** IRS allow the "decoupling" of credit risk and interest rate risk. This enables a more efficient capital allocation in the economy, as companies can choose their funding structure based on their comparative advantage (credit) and then adjust their interest rate exposure through derivatives.

5.8 Conclusions and Key Takeaways

- The floating leg always equals par regardless of the shape of the yield curve. This is not an approximation but an exact result from the telescoping property of discount factors: all pricing complexity of the swap lives in the fixed leg alone.
- The swap rate K^* is the weighted average of the forward rates implied by the discount curve, with weights proportional to $\delta_i \cdot Z(t_i)$. In an upward-sloping curve, short forward rates pull K^* below the long-maturity spot rate, explaining why the swap curve lies below the spot curve for long tenors.
- The FRA decomposition exposes the internal structure of the swap: periods where the forward rate exceeds K contribute positive value to the fixed payer, and vice versa. At $K = K^*$ these contributions cancel exactly, confirming the fair pricing condition. Note that the forward rates in each FRA must be computed under simple compounding, consistent with how LIBOR and SOFR rates are quoted in practice.
- The curve trade (pay fixed short, receive fixed long) is P&L neutral to parallel shifts when DV01s are matched. Profit and loss arise purely from changes in the slope of the yield curve, making it the canonical instrument for expressing a view on curve shape without directional rate exposure.

6 Project Summary and Further Research

What This Project Built

This project developed a complete, self-contained fixed income analytics library in Python, grounded in formal mathematical derivations and validated against real market data from the Federal Reserve. The five modules form a coherent chain:

1. **Module 1** established the mathematical foundations of bond pricing, from closed-form formulas to Newton-Raphson yield extraction and second-order risk approximations.
2. **Module 2** replaced the constant discount rate with a full term structure, building the yield curve from observable prices via bootstrapping and extending it to arbitrary maturities through three interpolation methods of increasing sophistication: linear, cubic spline and Nelson-Siegel-Svensson.
3. **Module 3** decomposed interest rate risk from a single parallel DV01 into a full Key Rate Duration vector, enabling granular hedging and non-parallel scenario analysis across a portfolio of bonds.
4. **Module 4** reduced the m -dimensional KRD vector to three orthogonal factor exposures via PCA, confirmed the Litterman-Scheinkman result empirically, and connected it to the NSS parametric model, showing that both frameworks discover the same underlying structure.
5. **Module 5** closed the cycle by pricing vanilla Interest Rate Swaps using the discount factors of Module 2, decomposing them into FRA portfolios, and exposing their sensitivity profile as a KRD DV01 vector that feeds directly into the risk framework of Modules 3 and 4.

The result is a project where each module is both mathematically derived in this document and numerically implemented in a corresponding Python class, with interactive notebooks that validate every result against real or realistic market data.

Virtues of the Approach

- **Theory-code correspondence:** every formula in this document has a direct implementation in `src/`, with identical notation. The document and the code are designed to be read side by side.
- **Validation by construction:** roundtrip tests (price $\rightarrow$ YTM $\rightarrow$ price), the KRD additivity property, the floating leg = par result and the FRA sum = 0 condition are all checked programmatically in every notebook run. The library cannot silently produce wrong results.
- **Real data:** all yield curves are built from U.S. Treasury data sourced from FRED, ensuring that the results reflect actual market conditions rather than stylized examples.
- **Modularity:** each class is self-contained and interoperable. A `DiscountCurve` built in Module 2 is consumed by `BondSensitivity` in Module 3, `PCARiskManager` in Module 4 and `VanillaIRS` in Module 5 without modification.

Limitations

- **Single-curve framework:** All modules assume a single discount curve for both projection and discounting. Post-2008 markets require a multi-curve framework where the floating leg is projected using a tenor-specific forward curve (e.g., EURIBOR 6M) and discounted using an OIS curve (SOFR or ESTR). The mathematical extension is straightforward but requires a second `DiscountCurve` object and a modified `fra_values()` method.
- **Vanilla instruments only:** The library prices plain vanilla bonds and swaps. It does not cover options, swaptions, caps and floors, callable bonds or any instrument with optionality, which require stochastic rate models (Hull-White, Black-Karasinski) beyond the deterministic curve framework of this project.
- **Deterministic curve:** The yield curve is treated as a deterministic function of time. A stochastic extension would model $r(t)$ as a stochastic process (e.g., Vasicek, CIR or HJM), enabling the pricing of interest rate derivatives with early exercise features.
- **No credit risk:** All instruments are assumed default-free. Incorporating credit spreads, survival probabilities and recovery rates would extend the framework to corporate bonds and credit default swaps (CDS).
- **Static PCA:** The PCA of Module 4 is estimated on a fixed historical window. A dynamic extension using rolling or exponentially weighted covariance matrices would allow the factor structure to adapt to changing market regimes, which is particularly important around central bank policy shifts.

Natural Extensions

Each limitation above is also a natural next step, ordered from most to least immediate:

1. **Multi-curve framework** (Module 2 extension): Implement separate projection and discount curves, calibrate basis spreads and reprice the swap portfolio under the post-2008 standard.
2. **Interest rate options** (Module 6): Implement caps, floors and European swaptions under Black's model, connecting the volatility surface to the implied forward rate distribution.
3. **Stochastic rate models** (Module 7): Implement Vasicek and Hull-White one-factor models, calibrate them to the observed yield curve and price Bermudan swaptions via trinomial trees or Monte Carlo simulation.
4. **Credit risk** (Module 8): Extend the discount curve to incorporate credit spreads, implement the Merton structural model and price Credit Default Swaps as a natural complement to the IRS framework.
5. **Dynamic risk management** (Module 9): Replace the static PCA with a Dynamic Nelson-Siegel model, implement Kalman filtering to track factor evolution in real time and back-test the factor immunization strategy across historical rate cycles.

The framework developed here is the foundation on which all these extensions build. Every new instrument will consume a `DiscountCurve`, expose a KRD DV01 vector and connect to the PCA risk engine, because that is how the fixed income world is organized, from the simplest Treasury bill to the most complex structured product.

References

- Ho, T. S. (1992). Key rate durations: Measures of interest rate risks. *The Journal of Fixed Income*, 2(2):29–44.
- Litterman, R. B., Scheinkman, J., and Weiss, L. (1991). Volatility and the yield curve. *The Journal of Fixed Income*, 1(1):49–53.
- Nelson, C. R. and Siegel, A. F. (1987). Parsimonious modeling of yield curves. *Journal of business*, pages 473–489.
- Svensson, L. E. (1994). Estimating and interpreting forward interest rates: Sweden 1992-1994.